UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

FACULTY OF ENGINEERING, BUILT ENVIRONMENT AND INFORMATION TECHNOLOGY

DEPARTMENT OF ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING
http://www.up.ac.za/eece

LAB BOOK

Compiled by

Dean Mills
13081650

Generated on

Monday 6th April, 2026

ISG@UP
INTELLIGENT SYSTEMS GROUP
http://isg.up.ac.za/

Contents

1 Catchup **3**
2026/03/26 . . . 3
Dataset Generator Refactor . . . 3
GAT . . . 5
Grouping Head Selection . . . 7
DEC (Deep Embedded Clustering) . . . 7
Slot Attention . . . 8
Graph Partitioning . . . 10
DETR — Rationale for Exclusion . . . 11
Experimental Results . . . 11
SA-DMoN (Skeleton-Aware Deep Modularity Networks) . . . 15
Removing Depth — Isolating the Grouping Problem . . . 17
SA-DMoN v1 — Skeleton-Aware Null Model . . . 19
SA-DMoN v2 — Diagnosing and Fixing Signal Cancellation . . . 21
Experiment 8 — Spectral Loss Removed Entirely . . . 25
Experiment 9 — COCO Zero-Shot Transfer, No Depth . . . 26
SCOT — Skeleton-Constrained Optimal Transport . . . 27
COP-Kmeans — Type-Constrained K-Means . . . 44
Type-Constrained Agglomerative Clustering . . . 48
Skeleton-Aware Spectral Clustering . . . 50
SA-GAT — Skeleton-Aware Graph Attention Network . . . 53
SA-GAT + SCOT — Combining the Two Contributions . . . 58
K Estimation — Removing the Ground Truth K Dependency . . . 60
Training Infrastructure — GUID-based Output Management . . . 63

Appendices **66**
Acronyms . . . 66

References **67**

1 Catchup

Thursday, 26 March 2026

This chapter covers everything that has changed since the last proper labbook entry. I've been bad about keeping up with this so this is a big dump of what happened, what worked, and what didn't.

Dataset Generator Refactor

The old code had separate files for everything — `virtual_dataloader.py`, `virtual_dataset.py`, `virtual_preprocessor.py` as well as `dataloader.py`, `dataset.py`, `preprocessor.py`, `depth_estimation.py`. The architecture has been updated to use a unified pipeline:

```
COCO data          Virtual data
    |                   |
coco_adapter.py    virtual_adapter.py
         |         |
        dataset.py          <-- single Dataset class
             |
        preprocessor.py     <-- single graph builder
             |
        dataloader.py       <-- single DataLoader
```

Two reasons for this:

- After the adapter it doesn't matter where the data came from, the rest of the pipeline is identical
- Can train on synthetic data first then swap to COCO without changing anything downstream

Extractor changes

Keypoint format changed from $[x, y, v]$ to $[x, y, z, v]$ per joint (51 $\rightarrow$ 68 values per person). Per-joint depth is strictly better than one depth value per person. z is world-space Euclidean distance from camera to joint in metres — consistent across all camera positions, matches MiDaS semantics after normalisation.

Visibility changed from $0/2$ to $0/1/2$ with raycast occlusion detection. Occluded joints are real training signal — marking them as invisible was losing information.

`calculate_distance_to_camera` was removed — redundant now that there is per-joint z.

Out-of-frame joints position changed from $[0.0, 0.0]$ to $[-1.0, -1.0]$. Zero is a real pixel location (top-left corner), -1 is an unambiguous sentinel meaning "no valid position". The GAT uses visibility $v = 0$ to ignore these joints during message passing but the position itself should also signal invalidity explicitly.

Renderer changes

Filenames changed to `<guid>_<n>.png` / `<guid>_<n>.json`. GUIDs eliminate ID collisions, the `_n` suffix allows filtering by person count without parsing JSON. `num_people` was added to image metadata for debugging and dataset statistics.

Virtual adapter

Loads a folder of `<guid>_n.png/<guid>_n.json` pairs and scans the folder to filter using min and max people. Converts from the per-image JSON (68 floats per person) to the unified output:

```
{
    "image":      tensor [3, H, W],
    "keypoints":  list of tensor [17, 4],  # [x, y, z, v]
    "img_id":     str,
    "num_people": int
}
```

No transforms, no normalisation, no MiDaS — just reading and reshaping. Everything else happens downstream.

COCO adapter

Loads COCO 2017 person keypoint annotations via pycocotools, filters by min/max person count the same way as the virtual adapter. MiDaS DPT_Hybrid is initialised once at construction time and used to estimate a depth map per image. Depth is then sampled at each keypoint's pixel location.

Converts from COCO's flat format (51 floats per person, COCO visibility 0/1/2) to the same unified output as the virtual adapter.

Visibility is handled explicitly:

COCO v	Position	Depth
0 (not labelled)	$[-1, -1]$	0
1 (occluded)	real x, y	MiDaS sampled
2 (visible)	real x, y	MiDaS sampled

Dataset

Single shared Dataset class that sits between the adapters and the rest of the pipeline. Accepts either adapter and applies a consistent transform so everything downstream sees the same tensor shapes regardless of data source.

- Image resized to fit within `target_size` × `target_size` preserving aspect ratio
- Symmetrically padded to exactly `target_size` × `target_size`
- Keypoint x, y scaled and shifted to match the new image dimensions
- z and v columns left untouched
- Joints with $v = 0$ (sentinel $-1, -1$) are not transformed

Dataloader

Wraps a PoseDataset with a custom collate function. Images stacked into a single tensor, keypoints kept as a list of lists since person count varies per image. `batch_size` is an explicit argument controlled from `config.yaml`.

Preprocessor

Single unified graph builder. Converts a DataLoader batch into PyTorch Geometric graphs ready for the GAT. One graph per image, all people's joints are nodes in the same graph.

Joints with $v = 0$ are excluded entirely — no position, no node. Everything else becomes a node with features:

Feature	Description
x_{norm}	x normalised to $[0, 1]$ by image size
y_{norm}	y normalised to $[0, 1]$ by image size
z_{norm}	depth normalised per-graph min-max over valid joints
v_{norm}	visibility normalised ($0 \to 0.0$, $1 \to 0.5$, $2 \to 1.0$)

Separate tensors per graph: `joint_types` ($[N]$ long, index 0–16 for GAT embedding layer) and `person_labels` ($[N]$ long, ground truth person assignment for training).

Edges are built using kNN with $k = 8$ on $(x_{\text{norm}}, y_{\text{norm}}, z_{\text{norm}})$ — depth is part of the proximity calculation so joints that are close in 3D get connected even if far apart in 2D.

Key decisions:

- Per-graph depth normalisation — MiDaS and metric depth have different absolute scales, normalising per-graph makes both sources comparable
- `joint_types` separate from features — lets the GAT learn a per-joint-type embedding independently of position
- kNN includes depth — richer connectivity than 2D-only

All pipeline tests passed — see `outputs/test_outputs/data`.

GAT

The GAT (Graph Attention Network) takes the graph of joints built by the preprocessor and produces a 128-dimensional embedding for each joint. The goal is for joints belonging to the same person to end up close together in this 128D space, and joints from different people to be far apart.

The architecture:

- Each joint starts as a 4-value feature vector $[x, y, z, v]$ plus a learned 32-dim embedding for its joint type (nose, left knee, etc.)
- Two GATv2 layers with 4 attention heads do message passing — each joint updates its representation by attending to its neighbours
- A final linear projection maps down to 128 dimensions

- L2 normalisation so all embeddings live on the unit hypersphere

The joint type embedding matters — a left knee should have different characteristics than a nose regardless of position, and keeping this separate from the spatial features lets the GAT learn that independently.

Training (isolation phase)

The GAT is trained in isolation using contrastive loss only. Pull same-person joint embeddings together, push different-person joint embeddings apart. No grouping head yet — just verifying the embedding space is structured correctly before adding the next component.

Config:

```
gat:
  num_joint_types: 17
  joint_embedding_dim: 32
  raw_feature_dim: 4
  hidden_dim: 64
  output_dim: 128
  num_layers: 2
  num_heads: 4
  dropout: 0.1
  use_layer_norm: true
  l2_normalize: true

loss:
  contrastive_margin: 0.5
```

Isolation test results

Four sanity checks passed:

- Forward pass — output shape $[N, 128]$
- L2 normalisation — all norms ≈ 1.0
- Loss computation — scalar, no NaN, ≥ 0
- Gradient flow — all parameters receive gradients

Two separability runs on a fixed scene with 5 people and 85 joints:

Metric	100 steps / 8 graphs		500 steps / 16 graphs	
	Before	After	Before	After
Silhouette	-0.043	-0.082	-0.030	-0.042
Distance ratio	1.006	1.108	0.992	1.020
kNN accuracy	0.630	0.983	0.596	0.985

kNN accuracy is the most meaningful number — 0.983 and 0.985 means almost every joint's 5 nearest neighbours in embedding space belonged to the same person. The distance ratio is noticeably better in the 100-step run (1.108 vs 1.020) which is expected — 16 graphs covers more diverse scenes making the task harder. Silhouette stays negative in both runs due to geometric compression on the unit hypersphere — not a useful metric here, reported for completeness only.

PCA projects the 128D embeddings down to 2D by finding the axes of maximum variance. Before training both plots show colours scattered and mixed with PCA variance explained around 87%. After 100 steps: five tight, clearly separated clusters with PCA variance explained at 99.17%. After 500 steps: same clean result at 97.56%.

t-SNE preserves local neighbourhood structure rather than global variance. Same-colour points clustering together in both after-training plots confirms the embedding space is well structured.

See `outputs/test_outputs/gat`.

Grouping Head Selection

GAT isolation verified. The next step was the grouping head — takes the embeddings from the GAT and produces the final joint-to-person assignment. Four candidates were considered:

- Deep Embedded Clustering (DEC)
- DETR (DEtection TRansformer)
- Graph partitioning
- Slot attention

DEC (Deep Embedded Clustering)

DEC takes embeddings and learns K cluster centres in the same embedding space. Each joint gets a soft assignment q based on distance to each centre using a Student's t-distribution. A sharpened target distribution p is computed from q and the model is trained to minimise KL divergence between them — pushing soft assignments toward harder, more confident ones over time.

Cluster centres are initialised with k-means++ before training. One important detail: p is updated every `update_interval` steps rather than every step. Updating every step causes cluster collapse — the model chases a moving target and all joints pile into one cluster by step 150. Periodic updates let the centres stabilise between target refreshes.

What I tried and why it kept failing

Attempt 1 — random GAT embeddings. With an untrained GAT all joints are roughly equidistant from all cluster centres, so q is nearly uniform across all K clusters. p is computed from q so $p \approx q$, which means $\mathrm{KL}(p\|q) \approx 0$ immediately — no loss, no gradient, no learning.

Attempt 2 — pre-trained GAT embeddings. With 200 steps of GAT pre-training, k-means++ initialised perfectly (accuracy 1.0 before any DEC training). But then DEC actively degraded:

```
Before training: accuracy = 1.0000

Step  25: accuracy=1.0000  loss=0.0661

...

Step 125: accuracy=0.5765  loss=0.0005

Step 150: accuracy=0.2000  loss=0.0002
```

This is cluster collapse — p was being recomputed from q every step, the model chased a moving target, and all joints piled into one cluster. Fixed with periodic p updates but a deeper problem remained: if the GAT is good there's nothing for DEC to do.

Attempt 3 — periodic updates, real scale (17 joints per person, 128D). Switched to fabricated embeddings (Gaussian blobs on the unit hypersphere with known ground truth) to control difficulty. Hit a hard ceiling:

```
Step 250: accuracy=0.7647  loss=0.0273
Step 300: accuracy=0.7647  loss=0.0272
...
Step 1000: accuracy=0.7647  loss=0.0274
```

0.7647 exactly — 4 out of 5 clusters correct, one pair always confused regardless of noise level or number of steps. This is the curse of dimensionality — 17 points in 128D is too sparse for the Student's t soft assignments to produce a meaningful gradient.

Attempt 4 — 50 joints per person. Bumped `n_per_person` to 50:

```
Before training: accuracy = 0.6200
Step 250: accuracy=0.9680
Step 300: accuracy=1.0000
```

DEC works — but only with 3× more points per cluster than the real problem has.

Isolation test results

5 clusters, 50 points per cluster, 128D, `update_interval=20`:

	Before training	After training
Easy (noise=0.05)	1.000	1.000
Medium (noise=0.1)	0.620	1.000

Accuracy measured with Hungarian matching since DEC cluster indices are arbitrary.

Why DEC isn't the right head

DEC is fully unsupervised — it never sees ground truth labels. It can only sharpen assignments that are already roughly correct, it has no signal to fix wrong ones. With 17 joints per person in 128D there is a hard ceiling at 0.8 regardless of tuning. If the GAT is doing its job, kNN accuracy on the embeddings is already 0.98–1.0 and k-means++ alone at inference time would give near-perfect assignments. DEC adds complexity and collapse risk without improving on the initialisation.

The right grouping head needs to learn from ground truth labels. Graph partitioning and slot attention both do this.

Slot Attention

Why slot attention

DEC failed at realistic scale because it is unsupervised. Slot attention is trained end-to-end with ground truth person assignments via Hungarian-matched cross entropy.

What it does

Slot attention is an iterative competitive attention mechanism. K slot vectors are initialised by sampling from a learned Gaussian distribution, one slot per person. Over several iterations each slot attends to all joints and competes for the ones it best represents. The competition is enforced by applying softmax over slots rather than over joints — each joint's attention weights sum to 1 across all slots, so if two slots compete for the same joint the better-matching one wins.

After iterative refinement the final slot vectors score each joint via dot product, producing logits $[N, K]$. Hungarian matching finds the optimal bijection between predicted slots and ground truth people, and cross entropy is computed on the matched assignments.

The architecture:

```
Inputs:  embeddings [N, D]  L2-normalised GAT embeddings
         k          int     number of people in this scene


Slots sampled from learned mu, sigma  ->  [K, D]


For num_iterations:
    norm(slots)   ->  queries  [K, D]
    norm(inputs)  ->  keys     [N, D]
                      values   [N, D]

    scores   =  queries . keys^T * scale     [K, N]
    softmax over slots  (competition)        [K, N]
    normalise within each slot               [K, N]
    updates  =  weighted sum of values       [K, D]
    slots    =  GRU(updates, slots)          [K, D]
    slots    =  slots + FF(LayerNorm(slots)) [K, D]

slot_keys  =  linear(slots)                 [K, D]
logits     =  embeddings . slot_keys^T      [N, K]
```

Loss: Hungarian matching between predicted slots and ground truth people, then cross entropy on matched assignments.

Config:

```
slot_attention:
  num_iterations: 7
```

```
slot_weight:    1.0
```

What went wrong during testing

Instability at lr=10^{-3}, 3 iterations — the model found the correct solution early (accuracy 1.0 at step 100) but then overshot and lost it, ending at 0.78. Slots reinitialise from the learned Gaussian every forward pass — with only 3 iterations they do not always converge, producing noisy gradient signal.

Fix 1 — more iterations. Increasing `num_iterations` from 3 to 7 stabilised the assignments within each forward pass and produced cleaner gradient signal.

Fix 2 — cosine LR schedule. A flat lr of 10^{-3} caused overshooting. A cosine schedule decaying from 10^{-3} to 10^{-5} allows fast early learning and fine-grained settling.

Isolation test results

5 clusters, 17 joints per person (real scale), 128D, 500 steps, cosine LR $10^{-3} \to 10^{-5}$:

	Before training	Best accuracy	Step reached
Easy (noise=0.05)	0.459	1.000	250
Hard (noise=0.15)	0.400	1.000	200

Both tests reach perfect accuracy at realistic joint counts — 17 joints per person, the same scale the real problem provides. DEC hit a hard ceiling of 0.8 at this scale and never improved.

Graph Partitioning

What it does

Graph partitioning frames the grouping problem as binary edge classification. For every pair of joints (i, j) the model predicts whether they belong to the same person. After thresholding, connected components on the resulting affinity graph recover the person groups without any explicit knowledge of K.

This is a fundamentally different framing from slot attention. Slot attention assigns joints to K pre-specified slots via competitive attention. Graph partitioning makes pairwise same/different decisions and lets the group structure emerge from the graph topology. K does not need to be specified at inference time — it falls out naturally from the number of connected components.

For each pair (i, j) the feature vector is:

$$f(i, j) = [e_i - e_j \parallel e_i \odot e_j] \in \mathbb{R}^{2D} \tag{1.1}$$

where e_i, e_j are L2-normalised GAT embeddings. The difference captures direction between the two points in embedding space; the element-wise product captures co-activation. The MLP maps $\mathbb{R}^{2D}$ to a scalar logit per pair.

Groups are recovered at inference by thresholding predicted affinities at 0.5 and finding connected components via union-find.

Loss is binary cross entropy over all $N(N-1)/2$ pairs. With 5 people $\times$ 17 joints there are 680 same-person pairs and 2890 different-person pairs — a 4:1 class imbalance. `pos_weight = n_neg / n_pos` (clamped at 10) upscales the positive class.

Two metrics tracked:

- **Edge F1** — precision/recall balance on the binary pair classification (accuracy is misleading under class imbalance)
- **Grouping accuracy** — end-to-end metric: thresholded edges → connected components → Hungarian matching against ground truth

Isolation test results

5 clusters, 17 joints per person (real scale), 128D, 500 steps, cosine LR $10^{-3} \rightarrow 10^{-5}$:

	Before training	Best F1	Best grouping acc	Step
Easy (noise=0.05)	F1=0.083 / acc=0.212	1.000	1.000	50
Hard (noise=0.15)	F1=0.235 / acc=0.200	1.000	1.000	50

Perfect F1 and grouping accuracy by step 50 on both tests, holding stable for the remaining 450 steps with loss decaying smoothly to near zero. The cleanest result of all three grouping heads tested.

Comparison across all grouping heads

	DEC	Slot attention	Graph partitioning
Supervision	Unsupervised	Supervised (CE)	Supervised (BCE)
Ceiling at 17 joints / 128D	0.8	1.0	1.0
Requires K at inference	Yes	Yes	No
Stability	Collapsed	Noisy	Very stable
Steps to converge	Never	∼200	∼50
End-to-end trainable	Weakly	Yes	Yes

The key practical advantage of graph partitioning is stability and convergence speed — direct binary supervision from 3570 labelled pairs per scene is dense and unambiguous. The key theoretical advantage over slot attention is that K does not need to be known at inference time.

DETR — Rationale for Exclusion

DETR was considered as a fourth grouping head but not pursued to isolation testing.

Architectural overlap with slot attention. DETR's decoder is a superset of slot attention — it adds self-attention between queries and a heavier feed-forward stack, but the core mechanism is the same iterative cross-attention competition. The additional complexity adds no meaningful capability for this problem.

The null object problem. DETR handles variable detection counts by padding predictions with a learned "no object" class. Resolving the null value instability encountered in the preliminary implementation requires careful loss masking and balanced sampling — engineering overhead with no benefit for this problem where the number of people per scene is bounded and known at training time.

The comparison is already complete without it. The four approaches evaluated (kNN on GAT embeddings, DEC, slot attention, graph partitioning) span the relevant design space: unsupervised vs supervised, slot-based vs pairwise, K-required vs K-free. DETR would not add a meaningfully distinct data point.

Experimental Results

Metrics

PGA (Pose Grouping Accuracy). The primary metric. For each scene, predicted person groups are matched to ground truth people via Hungarian matching — finding the optimal bijection between predicted and ground truth

labels. PGA is the fraction of joints assigned to the correct person under this optimal matching. A score of 1.0 means every joint was grouped correctly. This metric is permutation-invariant.

NMI (Normalised Mutual Information). Measures the quality of the GAT embeddings independently of the grouping head. K-means is run on the embeddings with ground truth K, and NMI measures how much information the resulting clusters share with the ground truth person labels. NMI is the same across all methods evaluated from the same checkpoint because it depends only on the embeddings, not the grouping head.

ARI (Adjusted Rand Index). A second embedding quality metric. Measures agreement between k-means clustering and ground truth, corrected for chance. Like NMI, it is the same across all heads from the same checkpoint.

Detection F1. Measures whether the method correctly counts the number of people in the scene. For kNN this is always 1.0 because ground truth K is provided. For slot attention K is also provided. For graph partitioning K is inferred from connected components — if the edge threshold splits or merges groups, F1 drops.

Per-joint accuracy. PGA broken down by COCO joint type. Reveals whether certain joints are harder to group correctly — typically extremities (ankles, wrists) which are more likely to be occluded or close to another person's joints.

Experiment 1 — Synthetic test set

All models trained on 400 virtual scenes (100 each at 2/3/4/5 people), evaluated on 100 held-out virtual scenes. Training: 50 epochs, cosine LR $10^{-3} \rightarrow 10^{-5}$, AdamW.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT)	0.994	0.012	0.987	0.987	1.000
kNN (partition GAT)	0.995	0.012	0.987	0.987	1.000
Graph partitioning	0.954	0.115	0.987	0.987	0.973
kNN (slot GAT)	0.994	0.015	0.988	0.987	1.000
Slot attention	0.858	0.143	0.988	0.987	0.935

Per-joint accuracy on synthetic data:

Joint	kNN	Graph partition	Slot attention
nose	1.000	0.955	0.852
left eye	1.000	0.955	0.852
right eye	1.000	0.955	0.854
left ear	1.000	0.955	0.854
right ear	1.000	0.955	0.856
left shoulder	1.000	0.955	0.861
right shoulder	1.000	0.955	0.864
left elbow	0.992	0.955	0.860
right elbow	0.984	0.947	0.852
left wrist	0.994	0.955	0.873
right wrist	0.975	0.952	0.853
left hip	1.000	0.955	0.863
right hip	1.000	0.955	0.863
left knee	1.000	0.955	0.868
right knee	0.996	0.953	0.864
left ankle	0.990	0.955	0.855
right ankle	0.984	0.941	0.842

On synthetic data kNN is the strongest method. The GAT with contrastive loss produces near-perfect embeddings (NMI 0.987, ARI 0.987) and k-means is sufficient to recover the correct groups. The learned grouping heads do not improve on this. The head loss going to near zero by epoch 36 for graph partitioning confirms that the edge classifier solved its task quickly and stopped contributing meaningful gradient signal — the GAT was already making the problem trivially easy.

Experiment 2 — COCO val2017 (zero-shot transfer)

The same trained checkpoints evaluated on COCO val2017 with no fine-tuning. MiDaS DPT_Hybrid provides per-joint depth estimates. 2346 images, all with at least one annotated person. This is a zero-shot sim-to-real transfer test — the models have never seen real images during training.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT)	**0.838**	0.194	0.754	0.692	1.000
kNN (partition GAT)	0.836	0.195	0.751	0.687	1.000
kNN (slot GAT)	0.828	0.202	0.740	0.675	1.000
Slot attention	0.788	0.236	0.740	0.675	0.920
Graph partitioning	0.752	0.266	0.751	0.687	0.802

Per-joint accuracy on COCO:

Joint	kNN	Graph partition	Slot attention
nose	0.832	0.748	0.786
left eye	0.818	0.733	0.778
right eye	0.819	0.741	0.779
left ear	0.796	0.697	0.744
right ear	0.807	0.709	0.761
left shoulder	0.835	0.746	0.780
right shoulder	0.838	0.747	0.786
left elbow	0.841	0.739	0.780
right elbow	0.833	0.748	0.783
left wrist	0.848	0.732	0.780
right wrist	0.831	0.740	0.787
left hip	0.837	0.745	0.770
right hip	0.835	0.747	0.769
left knee	0.833	0.742	0.767
right knee	0.816	0.741	0.772
left ankle	0.775	0.706	0.749
right ankle	0.777	0.714	0.748

Conclusions from Experiments 1–2

1. The contrastive loss alone is the dominant learning signal. Joint training with a grouping head did not improve the GAT embeddings. kNN on the contrastive-only GAT (0.838) is essentially identical to kNN on the jointly trained GATs (0.836, 0.828). The heads add complexity without improving the upstream representation.

2. The sim-to-real gap is the primary open problem. PGA drops from 0.994 to 0.838 on real data for kNN. NMI drops from 0.987 to 0.754. The per-joint accuracy on COCO shows ankles and ears consistently lowest — joints that are frequently occluded or at the edge of the image, exactly where synthetic data diverges most from real scenes.

3. Slot attention generalises better than graph partitioning on real data. On synthetic data slot attention trails kNN by 14 points. On COCO that gap closes to 5 points. Graph partitioning goes the other direction — its detection F1 collapses to 0.802 because the fixed threshold of 0.5 calibrated on synthetic data does not transfer to noisier real embeddings.

4. kNN with a good GAT is a strong and robust baseline. Given ground truth K, k-means on L2-normalised GAT embeddings is competitive with or better than both learned heads on both domains. It has perfect detection F1 by construction and lower variance in every experiment.

5. Learned heads become relevant as scene complexity increases. The relative improvement of slot attention on COCO suggests learned grouping provides a meaningful benefit when embeddings are noisier. Fine-tuning on real annotated data is the natural next step.

SA-DMoN (Skeleton-Aware Deep Modularity Networks)

What DMoN is

DMoN (Deep Modularity Networks) is a differentiable graph clustering method from Tsitsulin et al. (2023). It learns to partition a graph into communities by maximising a differentiable relaxation of the modularity objective from spectral graph theory.

Given a graph with adjacency matrix A, find a soft assignment matrix $S \in \mathbb{R}^{N \times K}$ where S_{ik} is the probability that node i belongs to cluster k. DMoN optimises three losses simultaneously:

Spectral modularity loss. The modularity matrix is:

$$B = A - \frac{dd^\top}{2m} \tag{1.2}$$

where d is the degree vector and $m = \frac{1}{2}\sum_i d_i$ is the total edge mass. The spectral loss is:

$$\mathcal{L}_{\text{spectral}} = -\frac{\text{Tr}(S^\top BS)}{2m} \tag{1.3}$$

Minimising this maximises the trace of $S^\top BS$, meaning nodes in the same cluster are more connected than expected under the null model.

Orthogonality regularisation. Encourages cluster assignments to be quasi-orthogonal:

$$\mathcal{L}_{\text{ortho}} = \left\| \frac{S^\top S}{\|S^\top S\|_F} - \frac{I_K}{\sqrt{K}} \right\|_F \tag{1.4}$$

Collapse prevention. Prevents the trivial solution of assigning all nodes to a single cluster:

$$\mathcal{L}_{\text{cluster}} = \frac{\sqrt{K}}{N} \left\| \sum_i S_i \right\|_F - 1 \tag{1.5}$$

Why DMoN for this problem

The previous experiments established that the GAT produces good embeddings (kNN accuracy 0.98+ on synthetic data), the sim-to-real gap (0.994 → 0.838 PGA) is the primary bottleneck, and supervised heads didn't improve the GAT embeddings.

The hypothesis behind DMoN is that the modularity loss provides a **domain-invariant structural prior**. Unlike contrastive loss or supervised CE which depend on learned visual features, modularity reasons about graph topology — are these nodes more connected to each other than expected? This topological signal should persist even when visual features degrade on out-of-distribution data.

Vanilla DMoN is fully unsupervised, which would put it in the same category as DEC. The contribution here is three domain-specific modifications.

Modification 1: Keypoint-type exclusivity regularisation

In standard graph clustering any node can go in any cluster. But in pose grouping each person has at most one keypoint of each type. DMoN has no mechanism to enforce this.

Define a type indicator matrix $M \in \mathbb{R}^{N \times T}$ where $M_{it} = 1$ if keypoint i is of type t (with $T = 17$ for COCO joints). Then $M^\top S \in \mathbb{R}^{T \times K}$ gives, for each cluster k and type t, the total soft assignment mass. The type exclusivity loss is:

$$\mathcal{L}_{\text{type}} = \|\text{ReLU}(M^\top S - 1)\|_F^2 \tag{1.6}$$

This penalises any cluster that accumulates more than one keypoint of the same type. It is fully differentiable and directly encodes a structural constraint of the problem that no general-purpose graph clustering method captures.

Modification 2: Supervised-modularity hybrid loss

Rather than discarding the modularity objective in favour of pure supervision, the approach combines both:

$$\mathcal{L} = \lambda_1 \cdot \mathrm{CE}(S, S_{\mathrm{gt}}) + \lambda_2 \cdot \mathcal{L}_{\mathrm{spectral}} + \lambda_3 \cdot \mathcal{L}_{\mathrm{cluster}} + \lambda_4 \cdot \mathcal{L}_{\mathrm{type}} + \lambda_5 \cdot \mathcal{L}_{\mathrm{ortho}} \tag{1.7}$$

where $\mathrm{CE}(S, S_{\mathrm{gt}})$ is Hungarian-matched cross entropy. The supervised CE handles the "what to group" signal during training. The modularity loss handles the "how to group structurally" signal that should transfer across domains.

Implementation

Standard DMoN uses a fixed-K MLP for assignments. This doesn't work for this pipeline where K varies per scene (2–5 people). The implementation uses a pool of $K_{\max}$ learned cluster centre vectors. For a scene with K people, the first K centres are selected. Soft assignments are computed via scaled dot product:

$$S = \mathrm{softmax}\left(\frac{h \cdot C_{:K}^{\top}}{\sqrt{H}}\right) \tag{1.8}$$

where $h = \mathrm{MLP}(e)$ maps GAT embeddings to a hidden space and $C \in \mathbb{R}^{K_{\max} \times H}$ is the learned centre pool. Hungarian matching in the loss makes this permutation-invariant.

Config:

```
dmon:
  hidden_dim: 256
  k_max: 10
  dropout: 0.0
  lambda_spectral: 0.1
  lambda_ortho: 0.1
  lambda_cluster: 0.1
  lambda_type: 1.0
  lambda_supervised: 1.0
```

The structural losses (spectral, ortho, cluster) are weighted at 0.1 relative to supervised CE and type exclusivity at 1.0. This ensures the supervised signal dominates early learning while the structural terms act as regularisers. With equal weights the ortho loss ($\sim$1.05) dominated the total loss and drowned out the CE gradient, preventing convergence.

Design decisions

Stochastic vs deterministic centres. The first implementation sampled cluster centres from a learned Gaussian each forward pass (same approach as slot attention). This caused severe instability — accuracy oscillated wildly between 0.3 and 0.7 because the same input produced different assignments on different forward passes. Slot attention overcomes this with 7 iterations of competitive refinement; without that iterative convergence, the stochasticity kills learning. The fix was switching to deterministic learned centre vectors from a fixed pool.

Loss weight balance. With all lambda values at 1.0, the ortho loss ($\sim$1.05) was roughly equal to the CE loss ($\sim$1.6) and neither moved meaningfully. Setting structural loss weights to 0.1 let CE dominate while keeping the modularity signal as a regulariser.

Isolation test results

5 clusters, 17 joints per person (real scale), 128D, 500 steps, cosine LR $10^{-3} \rightarrow 10^{-5}$:

	Before training	Best accuracy	Step reached
Easy (noise=0.05)	0.318	1.000	50
Hard (noise=0.15)	0.318	1.000	50

Perfect accuracy by step 50 on both tests. The spectral loss drops from $\sim$0 to -0.746 on easy, confirming modularity is being actively maximised. The type loss stays near zero ($\sim$0.001), which is correct since the fabricated data has exactly one joint of each type per person.

Updated comparison across all grouping heads

	DEC	Slot attention	Graph part.	SA-DMoN
Supervision	Unsupervised	Supervised	Supervised	Hybrid
Ceiling (17j/128D)	0.8	1.0	1.0	1.0
Requires K	Yes	Yes	No	Yes
Stability	Collapsed	Noisy	Very stable	Very stable
Steps to converge	Never	$\sim$200	$\sim$50	$\sim$50
Type exclusivity	No	No	No	Yes
Structural reg.	No	No	No	Spectral

SA-DMoN matches graph partitioning's convergence speed and stability while adding a structural regulariser (modularity) that should provide domain-invariant signal during transfer, and a pose-specific constraint (type exclusivity) that encodes skeleton semantics.

Removing Depth — Isolating the Grouping Problem

Why remove depth

The previous experiments showed kNN at 0.994 PGA on synthetic data. That's near-perfect — the problem was too easy. Depth (per-joint z from ground truth camera distance) was doing most of the work separating people in 3D space. With depth the GAT barely needs to learn anything beyond spatial proximity, and a grouping head has no room to add value over simple k-means.

This is also why DMoN's spectral loss was ineffective — it's trying to improve on an already-solved problem.

Removing depth creates a harder, more realistic baseline. On COCO the depth signal comes from MiDaS estimation (noisy, arbitrary scale) rather than ground truth metric distance, so the with-depth synthetic results were artificially inflated. Training without depth forces the GAT and grouping head to work with 2D position and visibility only — the same signal quality available on real images without a depth estimator.

A `use_depth` flag was added to `GATConfig` and threaded through the preprocessor, trainer, and evaluator. When `use_depth: false`: node features become $[x_{\text{norm}}, y_{\text{norm}}, v_{\text{norm}}]$ (3 values not 4), kNN edges are built on $[x_{\text{norm}}, y_{\text{norm}}]$ only, and the GAT `input_dim` is computed accordingly.

Experiment 3 — Synthetic test set, no depth

All models trained on virtual data without depth, evaluated on 100 held-out virtual scenes. Training: 50 epochs for kNN-only, 150 epochs for DMoN.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, depth)	0.994	0.012	0.987	0.987	1.000
kNN (contrastive, no depth)	**0.901**	**0.110**	**0.831**	**0.801**	**1.000**
kNN (DMoN GAT, no depth)	0.881	0.116	0.806	0.763	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962

Per-joint accuracy (no depth):

Joint	kNN (contrastive)	kNN (DMoN GAT)	DMoN head
nose	0.871	0.876	0.764
left eye	0.895	0.891	0.768
right eye	0.903	0.886	0.765
left ear	0.930	0.899	0.768
right ear	0.908	0.897	0.776
left shoulder	0.932	0.903	0.839
right shoulder	0.925	0.912	0.818
left elbow	0.934	0.904	0.828
right elbow	0.890	0.910	0.796
left wrist	0.910	0.901	0.775
right wrist	0.886	0.876	0.764
left hip	0.923	0.894	0.814
right hip	0.922	0.887	0.813
left knee	0.884	0.862	0.766
right knee	0.890	0.862	0.786
left ankle	0.850	0.804	0.755
right ankle	0.864	0.817	0.751

Analysis

1. Removing depth creates a meaningfully harder problem. kNN drops from 0.994 to 0.901. NMI drops from 0.987 to 0.831. Extremities suffer most — ankles drop to 0.850/0.864, consistent with the intuition that limbs from different people overlap more in 2D when depth can't separate them.

2. DMoN does not beat kNN — the vanilla spectral loss hurts. The DMoN head (0.785) is worse than kNN on the same embeddings (0.881), and kNN on the DMoN-trained GAT (0.881) is worse than kNN on the contrastive-only GAT (0.901). The vanilla spectral loss degrades performance in two ways: the head itself

groups worse than k-means, and backprop through the spectral loss pushes the GAT toward embeddings that satisfy the wrong clustering criterion.

3. The degree-based null model is the root cause. The modularity matrix $B = A - dd^\top/(2m)$ asks "are these keypoints more connected than their degree would predict?" For a kNN graph built on 2D position this is the wrong question. Two shoulders from different people that happen to be spatially close will be kNN neighbours — the degree null treats this as evidence for same-cluster. Meanwhile a left wrist and left ankle from the same person that are far apart in 2D get little credit for being grouped together.

4. This directly motivates the skeleton-aware null model. Instead of asking "connected more than degree predicts," the skeleton-aware null asks "connected more than spatial proximity and anatomical adjacency predict." Two nearby shoulders from different people become *expected* under the null rather than surprising.

SA-DMoN v1 — Skeleton-Aware Null Model

A new `sa_dmon.py` module was created alongside the existing `dmon.py`. The architecture is identical to vanilla DMoN — the only change is in the spectral loss, which replaces the degree-based null with the skeleton-aware null model.

The skeleton-aware null combines two terms:

Spatial proximity kernel P. For each pair of nodes:

$$P_{i,j} = \exp\left(-\frac{\|pos_i - pos_j\|^2}{2\sigma^2}\right) \tag{1.9}$$

where pos_i is the normalised 2D position $(x, y) \in [0,1]^2$. Sigma is learnable, stored as $\log(\sigma)$ for unconstrained optimisation, initialised at 0.2.

Anatomical type affinity T. A fixed 17×17 matrix derived from the COCO skeleton topology via BFS: directly connected joints get 1.0 (e.g. left_shoulder $\leftrightarrow$ left_elbow), two hops get 0.5, three hops get 0.25, further or unconnected get 0.0.

Combined null R. For keypoints i (type a) and j (type b):

$$R_{i,j} = T_{a,b} \cdot P_{i,j} \tag{1.10}$$

Normalised to match the total edge mass: $R_{\text{norm}} = R \cdot (2m/\sum R)$.

Modified modularity matrix:

$$B_{\text{pose}} = A - R_{\text{norm}} \tag{1.11}$$

The spectral loss maximises $\text{Tr}(S^\top B_{\text{pose}} S)/(2m)$, rewarding grouping joints that are connected beyond what spatial proximity and anatomical adjacency would explain.

Experiment 4 — SA-DMoN no depth, learnable sigma

Trained on virtual data without depth, 150 epochs.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
kNN (DMoN GAT, no depth)	0.881	0.116	0.806	0.763	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962
kNN (SA-DMoN GAT, no depth)	0.870	0.123	0.796	0.748	1.000
SA-DMoN head (no depth)	0.779	0.140	0.796	0.748	0.953

Result: sigma diverged to 341. The learned spatial kernel bandwidth exploded from the initial 0.2 to 340.9, making the spatial kernel $\exp(-d^2/(2 \cdot 341^2)) \approx 1.0$ for all pairs. With P uniform, R_{norm} collapses to the type affinity matrix broadcast uniformly — a constant null model with no spatial discrimination.

This happened because the spectral loss gradient pushes sigma in the direction that makes modularity easiest to maximise. A flat kernel creates a weak null model that the adjacency trivially exceeds. The optimiser found a degenerate solution.

Two contributing factors: (1) `lambda_spectral` at 0.1 means the spectral loss contributes ∼10% of the total gradient — with the *correct* skeleton-aware signal now, suppressing it means the new null model barely influences training; (2) sigma with no bounds allows the optimiser to take the path of least resistance.

Experiment 5 — SA-DMoN no depth, sigma clamped $[0.05, 0.5]$

Sigma clamped using `log_sigma.clamp()` in the forward pass. All other settings identical.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962
SA-DMoN (clamped σ, λ=0.1)	0.805	0.149	0.803	0.759	0.965

Clamping fixed the divergence — sigma settled at the upper bound of 0.5, confirming the optimiser is still trying to push it as high as possible. The SA-DMoN head improved slightly over vanilla DMoN (0.805 vs 0.785, +2 points). A small signal that the skeleton-aware null is doing something when forced to operate at a reasonable bandwidth. But the head still loses to kNN on the same embeddings (0.805 vs 0.878).

Experiment 6 — SA-DMoN no depth, sigma clamped, $\lambda_{\text{spectral}}$ = 1.0

Lambda_spectral increased from 0.1 to 1.0 — now equal weight with supervised CE. The hypothesis was that the skeleton-aware null provides correct structural signal that should improve when given more influence.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962
SA-DMoN (clamped, λ=0.1)	0.805	0.149	0.803	0.759	0.965
SA-DMoN (clamped, λ=1.0)	0.790	0.151	0.788	0.740	0.950

The hypothesis was wrong. Increasing lambda degraded performance across the board. Sigma remained pinned at 0.5. The louder spectral signal actively fought the contrastive loss — the GAT loss increased from ∼0.063 to ∼0.070 mid-training before recovering. At $\sigma = 0.5$ in $[0, 1]$ normalised space the spatial kernel is nearly uniform — amplifying the weight of a weak signal just amplifies noise.

The deeper issue is an **incentive misalignment**: the spectral loss always benefits from a weaker null model (easier to show modularity against), so the optimiser pushes sigma as high as the clamp allows regardless of lambda. The useful structural signal requires a *strong* null model (small sigma, tight spatial kernel), but the gradient points in the opposite direction.

Experiment 7 — SA-DMoN no depth, sigma = median pairwise distance

Sigma removed from the optimiser entirely. Instead computed per graph as the median pairwise Euclidean distance between all node positions, detached from the computation graph. This adapts to each scene and eliminates the incentive misalignment. 100 epochs, lambda_spectral back to 0.1.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962
SA-DMoN (clamped, λ=0.1)	0.805	0.149	0.803	0.759	0.965
SA-DMoN (median σ, λ=0.1)	0.770	0.146	0.791	0.736	0.945

The worst SA-DMoN result yet (0.770). Sigma went from too broad to too tight. The median pairwise distance averages 0.21, but kNN edges connect nodes at $\sim$0.067 distance — 3$\times$ closer than the median. The null model is calibrated to the global distance scale, not the edge distance scale. Most actual kNN edges fall well within the kernel peak, so $B_{\text{pose}} \approx A$ and the spectral loss provides no useful gradient.

Summary of sigma experiments

Sigma strategy	σ value	Problem	Head PGA
Learned, unconstrained	341 (diverged)	Null too weak ($P \approx 1$)	0.779
Learned, clamped $[0.05, 0.5]$	0.5 (hit upper)	Null still too weak	0.805
Learned, clamped, λ=1.0	0.5 (hit upper)	Louder wrong signal	0.790
Median pairwise distance	$\sim$0.21	Null too tight ($P \approx A$)	0.770

No sigma value has produced a spectral loss that meaningfully helps grouping. The core tension: the spectral loss needs a null model calibrated to the *edge formation scale* (where kNN connections actually form), but every approach so far has been either too broad (null too weak, trivially exceeded) or too tight (null too strong, approximates the adjacency itself).

The deeper problem is that the kNN graph doesn't have the community structure modularity was designed to detect. The graph is built on spatial proximity, but the "communities" (people) are defined by identity. Two joints from different people are kNN neighbours whenever people are close together — that's the norm, not the exception.

SA-DMoN v2 — Diagnosing and Fixing Signal Cancellation

The diagnosis

Looking back at the four sigma experiments, the problem is more specific than "modularity doesn't work for pose graphs." The problem is **signal cancellation between the adjacency and the null model**.

The adjacency matrix A is built from kNN on 2D spatial positions — it encodes "these keypoints are spatially close." The skeleton-aware null model R_{norm} also encodes "these keypoints are spatially close" (via the Gaussian kernel on 2D positions, modulated by anatomical affinity). When computing the modularity matrix:

$$B_{\text{pose}} = A - R_{\text{norm}}$$

the result is subtracting a spatial signal from a spatial signal. B_{pose} ends up as near-zero noise everywhere because A and R are highly correlated. There is nothing left for modularity to optimise. The degree-based null in vanilla DMoN at least encodes something different from the adjacency (degree distribution vs edge existence), so B has a non-trivial signal — even if it's the wrong signal for pose grouping.

This explains why every sigma strategy failed in the same way: the spatial kernel P and the spatial kNN adjacency A are both functions of the same 2D positions. Changing sigma just changes how closely R approximates A — too broad and R is weaker than A (trivially exceeded), too tight and R matches A (nothing left). There is no sigma where the difference is both non-trivial and informative.

The fix: decouple the graphs

The adjacency and the null model need to encode *different information* so that their difference is meaningful. The fix is to build the adjacency for modularity from GAT *embedding similarity* rather than spatial proximity, while keeping the spatial/skeletal null model unchanged.

The GAT still runs message passing on the original spatial kNN graph — that doesn't change. But for the modularity computation, a new adjacency is built from kNN in embedding space:

$$H = \mathrm{GAT}(A_{\text{spatial}}, X) \qquad \text{(message passing on spatial graph)}$$
$$A_{\text{feature}} = \mathrm{kNN}(H) \qquad \text{(build new graph from embeddings)}$$
$$B_{\text{pose}} = A_{\text{feature}} - R_{\text{norm}} \qquad \text{(modularity matrix)}$$

Now $B_{\text{pose}} = A_{\text{feature}} - R_{\text{spatial}}$ is asking a meaningful question: "which keypoints are connected by learned feature similarity *beyond* what spatial proximity and anatomy predict?" A left shoulder and left elbow from different people might be spatially close (high R_{norm}) but have dissimilar embeddings (low A_{feature}), giving a negative B_{pose} entry — evidence they should be in different clusters. Two keypoints from the same person that are far apart but have similar embeddings give a positive B_{pose} entry — the modularity objective rewards grouping them.

The feature adjacency is detached from the gradient — it is a fixed graph structure for each forward pass, built from the current embeddings but not differentiated through. This prevents the modularity loss from gaming the adjacency the way it gamed sigma.

The second fix: entropy-based type exclusivity

The original type exclusivity penalty $\|\mathrm{ReLU}(M^\top S - 1)\|_F^2$ has a problem with softmax assignments. With softmax, each keypoint distributes mass across all clusters. If there are 5 left elbows and 3 person clusters, each cluster might get $5/3 \approx 1.67$ soft mass for the left-elbow type — but that is the expected behaviour of softmax, not a real violation. The penalty fires constantly and becomes noisy gradient signal.

The fix replaces the absolute-mass threshold with an entropy minimisation. For each cluster k and joint type t, normalise the soft assignments of type-t keypoints to cluster k into a probability distribution, then minimise its entropy:

$$\mathcal{L}_{\text{type}} = \frac{1}{|\mathcal{P}|} \sum_{(k,t)\in\mathcal{P}} H\left(\frac{S[\text{type}_t, k]}{\sum S[\text{type}_t, k]}\right)$$

This says "for each cluster-type pair, the assignment should be dominated by one keypoint, not spread across many." Low entropy means one keypoint has most of the mass (correct — one person has one left elbow). High entropy means the mass is spread evenly (wrong — multiple left elbows competing for the same person slot). This is a softer and more gradient-friendly signal than the ReLU threshold.

Implementation

Both fixes are implemented in `sa_dmon_v2.py` with config flags for independent ablation:

- `use_feature_adjacency`: if true, build adjacency from kNN in embedding space for modularity; if false, use the spatial graph (original behaviour)
- `use_entropy_type_loss`: if true, use entropy-based type exclusivity; if false, use the original ReLU penalty
- `feature_knn_k`: number of neighbours for the feature adjacency (default 8)

Because the feature adjacency now encodes different information from the spatial null, $\lambda_{\text{spectral}}$ is set to 1.0 for the feature adjacency configs. The original SA-DMoN needed 0.1 because the spectral signal was wrong and had to be suppressed — with decoupled graphs, the spectral signal should be meaningful and can be given full weight.

The ablation table:

Config	Feature adj	Entropy type	$\lambda_{\text{spectral}}$
SA-DMoN v1 (existing baseline)	no	no (ReLU)	0.1
v2 feature adjacency only	yes	no (ReLU)	1.0
v2 entropy type loss only	no	yes	0.1
v2 full (both fixes)	yes	yes	1.0

The targets: SA-DMoN v1's best result was 0.805 PGA on synthetic data (clamped sigma). SCOT achieved 0.923. If the signal cancellation diagnosis is correct, the feature adjacency configs should show a meaningful jump above 0.805. Beating kNN at 0.901 would validate the decoupled architecture. Matching SCOT at 0.923 would mean the DMoN formulation is competitive with optimal transport when given the right graph structure.

Experiment — Feature adjacency only ($\lambda_{\text{spectral}} = 1.0$)

First attempt: decoupled feature adjacency with $\lambda_{\text{spectral}} = 1.0$ (the reasoning being that the signal should now be meaningful and can be given full weight). ReLU type loss (unchanged from v1).

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
SA-DMoN v1 best (clamped σ)	0.805	0.149	0.803	0.759	0.965
SA-DMoN v2 feat adj (λ=1.0)	0.772	0.142	0.803	0.750	0.928

Worse than v1. The head loss remained very high at epoch 100 ($\sim$0.95) compared to v1's head loss at the same point ($\sim$0.15). With $\lambda_{\text{spectral}} = 1.0$, the spectral loss on the feature adjacency was dominating the gradient and not converging. The feature adjacency changes every forward pass as the embeddings evolve during training — at full weight, the spectral loss was chasing a moving target, the same instability that killed learned sigma.

Experiment — Feature adjacency only ($\lambda_{\text{spectral}} = 0.1$)

Dropped $\lambda_{\text{spectral}}$ back to 0.1 to let the spectral term act as a regulariser rather than a dominant objective.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
SA-DMoN v1 best (clamped σ)	0.805	0.149	0.803	0.759	0.965
SA-DMoN v2 feat adj (λ=1.0)	0.772	0.142	0.803	0.750	0.928
SA-DMoN v2 feat adj (λ=0.1)	0.786	0.145	0.794	0.745	0.959

Better than $\lambda = 1.0$ but still below v1's 0.805. The head loss was still high at epoch 100 ($\sim$1.17). The GAT embeddings were slightly degraded (NMI 0.794 vs 0.803 for v1), and detection F1 at 0.959 shows the head is getting person count wrong more often than v1 (0.965).

The feature adjacency fix does not improve performance at either lambda value. The signal cancellation diagnosis was partially correct — decoupling the graphs does produce a non-trivial B_{pose} — but the resulting modularity

signal is not useful for improving grouping. The feature adjacency built from GAT embeddings encodes the same person-identity signal that the supervised CE already provides directly. The spectral loss on A_{feature} is redundant with CE, adding noise without new information.

Experiment — Entropy type loss only (spatial adjacency)

Entropy-based type exclusivity with the original spatial adjacency and $\lambda_{\text{spectral}} = 0.1$. This isolates Fix 2 — does a better type loss formulation help even when the spectral signal is still cancelled?

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
SA-DMoN v1 best (clamped σ)	0.805	0.149	0.803	0.759	0.965
SA-DMoN v2 entropy only	0.802	0.141	0.806	0.767	0.961

Essentially neutral — 0.802 vs 0.805. The entropy type loss produces smoother training (head loss drops steadily from 1.41 to 0.19, much cleaner than the feature adjacency runs) but does not change the final accuracy. The entropy formulation is a better-behaved gradient signal than the ReLU threshold but both converge to the same ceiling.

Experiment — Full v2 (both fixes)

Both fixes combined: feature adjacency + entropy type loss, $\lambda_{\text{spectral}} = 0.1$.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
SA-DMoN v1 best (clamped σ)	0.805	0.149	0.803	0.759	0.965
SA-DMoN v2 full	0.784	0.150	0.795	0.749	0.936

Worse than v1 by 2.1 points. Combining both fixes does not produce a synergy — the feature adjacency degrades performance regardless of which type loss is used.

SA-DMoN v2 ablation summary

SA-DMoN variant	Feature adj	Entropy type	PGA	vs v1 best
v1 best (clamped σ, λ=0.1)	no	no (ReLU)	0.805	—
v1 no spectral	no	no (ReLU)	0.794	−1.1
v2 feat adj (λ=1.0)	yes	no (ReLU)	0.772	−3.3
v2 feat adj (λ=0.1)	yes	no (ReLU)	0.786	−1.9
v2 entropy only	no	yes	0.802	−0.3
v2 full (both)	yes	yes	0.784	−2.1

Table 1.1: Complete SA-DMoN v2 ablation. No variant improves on v1.

Conclusion: the DMoN head architecture is the bottleneck

Six v1 experiments and four v2 experiments all converge to the same 0.77–0.81 PGA range regardless of:

- What the spectral loss measures (degree null, spatial null, feature null, or nothing)
- How sigma is handled (learned, clamped, fixed, or removed entirely)
- What type of adjacency is used (spatial kNN or embedding kNN)
- How type exclusivity is formulated (ReLU threshold or entropy minimisation)

The consistent ∼8 point gap between the head and kNN on the same embeddings is not a loss problem — it is an **assignment mechanism problem**. The DMoN head computes soft assignments via a single-pass dot product between encoded node features and learned cluster centres, followed by softmax. This is fundamentally weaker than k-means, which iteratively refines centroids to minimise within-cluster variance. The single-pass dot product has no iterative refinement, no convergence guarantee, and distributes mass across all clusters via softmax even when the correct assignment is unambiguous.

This is the architectural insight that motivated the pivot to SCOT. SCOT replaces the soft assignment mechanism with Sinkhorn optimal transport, which is iterative (10 iterations), globally optimal (within the entropy-regularised space), and incorporates hard type constraints that the DMoN softmax cannot express. The 0.923 PGA that SCOT achieves — compared to the 0.805 ceiling of DMoN — is not because SCOT has a better loss function. It is because SCOT has a better assignment mechanism.

Experiment 8 — Spectral Loss Removed Entirely

Four sigma strategies all failed. The hypothesis shifted: perhaps the spectral loss itself is the problem, not its parameterisation. Setting `lambda_spectral: 0.0` isolates the head architecture from the conflicting spectral gradient. 100 epochs.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, no depth)	0.901	0.110	0.831	0.801	1.000
DMoN head (no depth)	0.785	0.140	0.806	0.763	0.962
SA-DMoN (clamped, λ=0.1)	0.805	0.149	0.803	0.759	0.965
SA-DMoN (median σ)	0.770	0.146	0.791	0.736	0.945
SA-DMoN (no spectral)	**0.794**	**0.143**	**0.795**	**0.748**	**0.963**

The best SA-DMoN head result so far (0.794), confirming that the spectral loss was actively hurting performance. The GAT embeddings are also the least degraded of any SA-DMoN experiment.

However, the head still does not beat kNN on the same embeddings. This pattern has been consistent across every experiment:

Experiment	Head PGA	kNN (same GAT)	Gap
Vanilla DMoN	0.785	0.881	−9.6
SA-DMoN (clamped σ, λ=0.1)	0.805	0.878	−7.3
SA-DMoN (clamped σ, λ=1.0)	0.790	0.867	−7.7
SA-DMoN (median σ)	0.770	0.865	−9.5
SA-DMoN (no spectral)	0.794	0.871	−7.7

The head is consistently ∼8 points below kNN regardless of spectral loss configuration. This gap is the head architecture's ceiling — the soft assignment via learned cluster centres + softmax is a weaker grouping mechanism than k-means on good L2-normalised embeddings.

The spectral loss question is now answered: it hurts in every form tested. The remaining question is whether a different graph-structural loss (normalised cut) can provide useful signal where modularity could not, or whether the assignment mechanism itself is the bottleneck.

Experiment 9 — COCO Zero-Shot Transfer, No Depth

All previous no-depth experiments were evaluated on synthetic data only. Before trying more loss engineering, I tested what I had on real data.

Method	Synthetic	COCO	Transfer gap
kNN (with depth)	0.994	0.838	−15.6
kNN (no depth)	**0.901**	**0.879**	−2.2
SA-DMoN head (no spectral, no depth)	0.794	0.817	+2.3

Full COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive, with depth)	0.838	0.194	0.754	0.692	1.000
kNN (contrastive, no depth)	**0.879**	**0.170**	**0.837**	**0.782**	**1.000**
kNN (SA-DMoN GAT, no depth)	0.864	0.179	0.810	0.749	1.000
SA-DMoN head (no spectral, no depth)	0.817	0.222	0.810	0.749	0.907

Per-joint accuracy on COCO (no depth):

Joint	kNN (no depth)	kNN (SA-DMoN GAT)	SA-DMoN head
nose	0.898	0.874	0.829
left eye	0.889	0.863	0.824
right eye	0.898	0.871	0.824
left ear	0.854	0.840	0.784
right ear	0.879	0.857	0.799
left shoulder	0.864	0.855	0.800
right shoulder	0.884	0.858	0.804
left elbow	0.852	0.845	0.800
right elbow	0.871	0.853	0.805
left wrist	0.854	0.852	0.804
right wrist	0.865	0.842	0.808
left hip	0.877	0.864	0.804
right hip	0.883	0.858	0.805
left knee	0.873	0.848	0.812
right knee	0.861	0.845	0.811
left ankle	0.833	0.814	0.790
right ankle	0.834	0.810	0.800

Analysis

1. Removing depth dramatically improves COCO transfer. kNN PGA jumped from 0.838 (with depth) to 0.879 (no depth) — a +4.1 point improvement on real data. The sim-to-real gap collapsed from 15.6 points to 2.2 points. NMI improved from 0.754 to 0.837, ARI from 0.692 to 0.782. The with-depth model learned to rely on clean ground-truth depth from synthetic data, then received noisy MiDaS inverse depth at inference on COCO. Removing depth forces the model to use only 2D position and visibility — signals that transfer cleanly with no domain gap.

This is the single largest improvement in COCO performance across all experiments in this project.

2. The SA-DMoN head improves on COCO vs synthetic. The head scored 0.794 on synthetic but 0.817 on COCO — a +2.3 point increase. This is the opposite of kNN, which drops from 0.901 to 0.879. The head is learning something that transfers better than the training distribution, likely because the type exclusivity constraint and supervised assignment mechanism are less sensitive to distributional shift than k-means on embeddings.

3. The head still does not beat kNN on COCO. SA-DMoN head at 0.817 vs kNN at 0.879 — a 6.2 point gap. The gap shrank from 7.7 points on synthetic to 6.2 on COCO, but the head does not close it.

4. Joint training with the head still degrades the GAT. kNN on the SA-DMoN-trained GAT (0.864) is 1.5 points below kNN on the contrastive-only GAT (0.879).

Updated conclusions

The most impactful finding from this project is not SA-DMoN — it is that **depth hurts sim-to-real transfer**. Ground truth synthetic depth and MiDaS estimated depth are different enough that the model learns to rely on a signal that degrades at inference on real data. A 2D-only model transfers better.

The SA-DMoN head with type exclusivity shows a promising transfer property (improving on COCO vs synthetic) but does not overcome the fundamental architectural gap with k-means. The head's soft assignment via learned cluster centres is a weaker grouping mechanism than iterative k-means on well-separated L2-normalised embeddings.

SCOT — Skeleton-Constrained Optimal Transport

Why pivot from DMoN

The SA-DMoN experiments exhaustively showed that modularity-based graph clustering is the wrong formulation for pose grouping. The kNN graph is built on spatial proximity, but the "communities" (people) are defined by identity — two joints from different people are kNN neighbours whenever people are close together, which is the norm not the exception. No null model fixes this because modularity on a spatial kNN graph is asking the wrong question.

The deeper insight from the SA-DMoN experiments is that type exclusivity — the constraint that a person has at most one keypoint of each type — was the one consistently useful modification. But in DMoN it was a soft penalty fighting against a clustering objective that didn't naturally want it. The question became: is there a formulation where type exclusivity is a hard architectural constraint rather than a soft loss term?

The formulation

Pose grouping is not a clustering problem — it is a **structured assignment problem**. Clustering says "put similar things together." Assignment says "fill specific slots with specific items under constraints." Every person

is a template with 17 typed slots (one per COCO joint type), and each detected keypoint can only fill a slot matching its type.

Given N detected keypoints and K people, define $K \times 17$ target slots. The transport plan $T \in \mathbb{R}^{N \times (K \cdot 17)}$ assigns keypoints to slots.

The cost matrix encodes the skeleton awareness:

$$C_{i,(k,t)} = \begin{cases} \|h_i - p_k\|^2 & \text{if type}(i) = t \\ +\infty & \text{if type}(i) \neq t \end{cases}$$

where h_i is the encoded GAT embedding for keypoint i and p_k is a learned prototype for person k. The infinity costs mean that the Sinkhorn solver **cannot** assign a left elbow to a right knee slot. It is not penalised — it is structurally impossible.

The transport plan is computed via differentiable Sinkhorn-Knopp iterations in log domain:

- Source marginal: each keypoint assigned to exactly one slot (row sum = 1)
- Target marginal: each person-type slot gets at most one keypoint (column sum ≤ 1)
- A slack column absorbs unmatched slots (handles missing/occluded joints)

Person-level logits are obtained by summing the transport plan over type slots per person: $\text{logits}_{i,k} = \sum_t T_{i,(k \cdot 17+t)}$. Training uses Hungarian-matched cross entropy on these logits — the same supervised signal as slot attention.

Why this is novel

Standard optimal transport for assignment (e.g. OTA for object detection) uses flat targets — anchors assigned to objects with no typed structure. SCOT has a two-level structure: person level and keypoint-type level. The target is not just K persons but $K \times 17$ typed slots. This hierarchical typed target is specific to the pose problem.

The same-type exclusivity is now impossible to violate by construction rather than discouraged by a penalty. Two left elbows competing for the same person will be forced into different person slots because there is only one left-elbow slot per person. This is fundamentally different from DMoN's $\|\text{ReLU}(M^\top S - 1)\|_F^2$ which can only discourage violations after they occur.

Implementation

The SCOT head (`ot_head.py`) follows the same pattern as other heads:

- Node encoder: Linear(D, H) $\rightarrow$ SELU — maps GAT embeddings to assignment space
- Person prototype pool: $K_{\max}$ learned vectors, first K selected per scene
- Cost matrix: squared distance between encoded nodes and prototypes, with 10^6 cost for type mismatches
- Sinkhorn iterations (10, temperature $\tau = 0.1$) in log domain for numerical stability
- Slack column with capacity $\max(N - K \cdot 17, 1)$ for unmatched slots

Config:

```
scot:
  hidden_dim: 256
  k_max: 10
  sinkhorn_iters: 10
  sinkhorn_tau: 0.1
```

No spectral loss, no null model, no sigma parameter. The only loss is contrastive (GAT) + Hungarian-matched cross entropy (head).

Experiment 10 — SCOT no depth, synthetic test set

Trained on virtual data without depth, 100 epochs. Same GAT config as all previous experiments.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT, no depth)	0.901	0.110	0.831	0.801	1.000
kNN (SCOT GAT, no depth)	0.886	0.117	0.803	0.767	1.000
SCOT head (no depth)	**0.923**	**0.105**	**0.803**	**0.767**	**0.997**
Previous best heads for comparison:					
SA-DMoN (no spectral)	0.794	0.143	0.795	0.748	0.963
Graph partitioning (with depth)	0.954	0.115	0.987	0.987	0.973
Slot attention (with depth)	0.858	0.143	0.988	0.987	0.935

Table 1.2: Synthetic test set results. SCOT is the first head to beat kNN.

SCOT beats kNN by 3.7 points (0.923 vs 0.886). This is the first time any grouping head has outperformed kNN on GAT embeddings in this project. Every previous head — DEC, slot attention, graph partitioning, vanilla DMoN, and all SA-DMoN variants — scored below kNN.

Per-joint accuracy shows the hard type constraint is doing real work on the hardest joints:

Joint	kNN (SCOT GAT)	SCOT head
nose	0.884	0.895
left eye	0.893	0.907
right eye	0.895	0.878
left ear	0.913	0.902
right ear	0.903	0.915
left shoulder	0.925	0.950
right shoulder	0.918	0.950
left elbow	0.914	0.965
right elbow	0.886	0.931
left wrist	0.902	0.939
right wrist	0.867	0.891
left hip	0.888	**0.968**
right hip	0.891	0.944
left knee	0.877	0.947
right knee	0.880	0.919
left ankle	0.823	0.897
right ankle	0.808	0.896

Table 1.3: Per-joint accuracy on synthetic test set. SCOT improves most on extremities.

The biggest improvements are on the joints that were hardest for kNN: left ankle +7.4, right ankle +8.8, left hip +8.0, left knee +7.0. These are the joints most likely to be spatially close to another person's joints in 2D — exactly where the type constraint prevents mis-assignment.

Experiment 11 — SCOT no depth, COCO zero-shot transfer

Same checkpoint evaluated on COCO val2017 with no fine-tuning.

Method	Synthetic	COCO	Transfer
kNN (contrastive, no depth)	0.901	0.879	−2.2
kNN (SCOT GAT)	0.886	0.877	−0.9
SCOT head	**0.923**	**0.862**	−6.1
SA-DMoN (best, no spectral)	0.794	0.817	+2.3

Table 1.4: Sim-to-real transfer comparison.

Full COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT, with depth)	0.838	0.194	0.754	0.692	1.000
kNN (contrastive GAT, no depth)	0.879	0.170	0.837	0.782	1.000
kNN (SCOT GAT, no depth)	0.877	0.172	0.835	0.779	1.000
SCOT head (no depth)	0.862	0.174	0.835	0.779	0.988

Table 1.5: COCO val2017 results.

Per-joint accuracy on COCO:

Joint	kNN (no depth)	kNN (SCOT GAT)	SCOT head
nose	0.892	0.898	0.846
left eye	0.885	0.889	0.818
right eye	0.888	0.898	0.797
left ear	0.848	0.854	0.766
right ear	0.873	0.879	0.733
left shoulder	0.865	0.864	**0.879**
right shoulder	0.885	0.874	**0.885**
left elbow	0.860	0.852	0.830
right elbow	0.857	0.871	0.830
left wrist	0.857	0.854	0.804
right wrist	0.854	0.865	0.802
left hip	0.883	0.877	**0.878**
right hip	0.880	0.883	**0.878**
left knee	0.868	0.873	0.834
right knee	0.865	0.861	0.832
left ankle	0.839	0.833	0.794
right ankle	0.835	0.834	0.790

Table 1.6: Per-joint accuracy on COCO. SCOT beats kNN on shoulders and hips (bold).

Analysis

1. SCOT is the first head to beat kNN on synthetic data. At 0.923 vs 0.886, the 3.7 point margin is substantial. The hard type constraint via the OT cost matrix is what makes the difference — every previous head used either no type constraint (slot attention, graph partitioning) or a soft penalty (DMoN/SA-DMoN type loss). Making type exclusivity structurally impossible to violate rather than penalised is the key architectural insight.

2. The SCOT-trained GAT barely degrades. kNN on SCOT GAT (0.886) vs contrastive-only GAT (0.901) is a 1.5 point gap — the same as SA-DMoN's best and much less than vanilla DMoN (2.0 points). The SCOT loss

cooperates with contrastive learning because both objectives want the same thing: same-person embeddings close, different-person embeddings far. DMoN's spectral loss actively fought the contrastive objective.

3. Training is dramatically cleaner than DMoN. Head loss drops smoothly from 1.17 to 0.02 over 100 epochs. No oscillation, no fighting between objectives, no sigma divergence. The best checkpoint is at epoch 30 (val PGA 0.905) — the model learns fast because the type constraint removes a large portion of the search space that other heads waste capacity exploring.

4. On COCO, SCOT (0.862) is the best head result by far but does not beat kNN (0.877). Previous best head on COCO was SA-DMoN at 0.817. SCOT beats that by 4.5 points. The gap to kNN on COCO (1.5 points) is the smallest of any head tested.

5. The COCO per-joint pattern reveals where transfer fails. SCOT beats kNN on torso joints (shoulders, hips) where the type constraint provides the most value — these are the joints most likely to have same-type neighbours from other people. But SCOT loses on head joints (eyes 0.797–0.818 vs 0.885–0.898, ears 0.733–0.766 vs 0.848–0.873). Face keypoints cluster tightly in 2D and all face types are distinct already, so the type constraint adds nothing there. The prototype-based cost function struggles when embeddings are noisy for these small, densely packed joints on real images.

6. The sim-to-real gap is larger for SCOT (-6.1) than kNN (-2.2). SCOT's synthetic advantage (0.923 vs 0.886) does not fully transfer — the head's learned cost function and Sinkhorn transport plan are more sensitive to distributional shift than simple k-means. This suggests the next step is either fine-tuning on a small amount of real data or making the cost function more robust to embedding noise.

Summary across all heads

Head	Synthetic PGA	COCO PGA	Beats kNN?	Type constraint
kNN (baseline)	0.901	0.879	—	None
DEC	0.8 ceiling	—	No	None
Slot attention	0.858	0.788	No	None
Graph partitioning	0.954	0.752	No	None
Vanilla DMoN	0.785	—	No	Soft penalty
SA-DMoN (best)	0.794	0.817	No	Soft penalty
SCOT	**0.923**	**0.862**	**Yes (synthetic)**	**Hard constraint**

Table 1.7: Comparison across all grouping heads tested in this project.

The progression tells a clear story: soft type penalties (DMoN/SA-DMoN) cannot overcome the fundamental limitations of clustering-based formulations. Hard type constraints via structured optimal transport (SCOT) reframe the problem correctly — as assignment rather than clustering — and achieve the first head-beats-kNN result in this project.

Qualitative example — COCO val2017

An end-to-end demo was built (`demo.py`) that takes a COCO image, runs the full pipeline (COCO adapter → dataset → preprocessor → GAT → SCOT), and draws predicted skeletons coloured by person assignment. Figure 1.1 shows a challenging 3-person scene with heavy occlusion.

Figure 1.1: End-to-end demo on a COCO val2017 image with 3 heavily overlapping people. Left: ground truth person assignments. Centre: SCOT prediction (PGA=0.656). Right: kNN prediction (PGA=0.594). Skeleton edges are coloured by predicted person assignment.

This is a hard scene — three people seated close together with significant occlusion, exactly the scenario where pose grouping is most difficult. Neither method achieves high PGA, but SCOT outperforms kNN by 6.2 points (0.656 vs 0.594).

The failure mode for kNN is visible in the right panel: the seated person in the front and the person behind them are merged — their body joints are mixed across the red and green clusters. kNN groups by embedding similarity, and when people overlap in 2D their embeddings are similar enough that k-means cannot separate them cleanly.

SCOT does better because the type constraint is active. The two overlapping people each have a left shoulder, left hip, left knee, etc. SCOT forces these same-type joints into different person slots — it is structurally impossible for two left shoulders to be assigned to the same person. The blue person (seated front) is more coherent in the SCOT panel than in kNN.

This illustrates the core argument for SCOT over clustering-based approaches: in crowded scenes with spatially overlapping people, the skeleton type structure provides disambiguating signal that embedding similarity alone cannot. The type constraint does not help in easy scenes (well-separated people) or for joints where types are already distinct (face keypoints), but it provides a meaningful advantage in exactly the hard cases where pose grouping matters most.

Experiment 12 — SCOT with depth

To complete the comparison, SCOT was trained with depth enabled (`use_depth: true`). This uses the same GAT configuration as the original experiments from Experiments 1–2: node features are $[x, y, z, v]$ and kNN edges are built in 3D space. The question: does SCOT's type constraint add value when depth already makes the problem easy?

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT, depth)	0.994	0.022	0.990	0.988	1.000
SCOT head (depth)	0.901	0.134	0.990	0.988	0.969
kNN (contrastive GAT, no depth)	0.901	0.110	0.831	0.801	1.000
SCOT head (no depth)	0.923	0.105	0.803	0.767	0.997

Table 1.8: SCOT with and without depth on synthetic test set.

Synthetic results. With depth, SCOT (0.901) does not beat kNN (0.994) — a 9.3 point gap. This is the opposite of the no-depth result where SCOT beat kNN by 3.7 points. Depth makes the problem so easy for kNN that the embeddings are near-perfect (NMI 0.990) and k-means trivially recovers the correct groups. There are no type collisions to resolve because people are already well-separated in 3D. The type constraint cannot add value when the problem is already solved, and the Sinkhorn approximation is less precise than exact k-means on clean embeddings.

The training dynamics confirm this: the GAT loss drops to 0.002 (vs 0.050 without depth) and the head loss to 0.006 (vs 0.024). The model converges much faster but to a worse solution than kNN because the transport plan introduces noise that k-means does not have.

Method	PGA	Std	NMI	ARI	Det F1
kNN (contrastive GAT, depth)	0.839	0.194	0.758	0.696	1.000
SCOT head (depth)	0.812	0.210	0.758	0.696	0.982
kNN (contrastive GAT, no depth)	0.879	0.170	0.837	0.782	1.000
SCOT head (no depth)	0.862	0.174	0.835	0.779	0.988

Table 1.9: COCO val2017 results with and without depth.

COCO transfer results. Per-joint accuracy on COCO with depth:

Joint	kNN (depth)	SCOT (depth)	SCOT (no depth)
nose	0.845	0.799	0.846
left eye	0.827	0.778	0.818
right eye	0.831	0.759	0.797
left ear	0.797	0.721	0.766
right ear	0.821	0.710	0.733
left shoulder	0.842	0.819	0.879
right shoulder	0.842	0.823	0.885
left elbow	0.835	0.792	0.830
right elbow	0.841	0.788	0.830
left wrist	0.839	0.776	0.804
right wrist	0.843	0.774	0.802
left hip	0.827	0.817	0.878
right hip	0.827	0.816	0.878
left knee	0.820	0.784	0.834
right knee	0.811	0.777	0.832
left ankle	0.764	0.742	0.794
right ankle	0.770	0.737	0.790

Table 1.10: Per-joint accuracy on COCO. No-depth SCOT beats with-depth SCOT on every joint.

Analysis. **1. Depth hurts SCOT on COCO even more than it hurts kNN.** SCOT drops from 0.862 (no depth) to 0.812 (with depth) on COCO — a 5.0 point penalty. kNN's depth penalty is 4.0 points (0.879 → 0.839). The depth domain gap compounds with SCOT's existing transfer sensitivity because the Sinkhorn transport plan is more affected by noisy features than k-means.

2. No-depth is strictly better for COCO transfer regardless of method. The complete ranking on COCO:

Method	COCO PGA
kNN (no depth)	**0.879**
SCOT (no depth)	0.862
kNN (with depth)	0.839
SCOT (with depth)	0.812

Table 1.11: All methods ranked by COCO PGA.

No-depth beats with-depth by ∼4–5 points across the board. This confirms the depth finding from Experiment 9: ground truth synthetic depth and MiDaS estimated depth are different enough that the model learns to rely on a signal that degrades at inference on real data.

3. SCOT's advantage only exists when the problem is hard. With depth, kNN at 0.994 leaves no room for improvement — the type constraint is solving a problem that does not exist. Without depth, kNN drops to 0.901 and the type constraint provides genuine value (SCOT 0.923). This is the correct interpretation of SCOT: it is not a universally better grouping method, it is a method that adds value specifically when spatial proximity alone is insufficient to separate people.

4. Per-joint accuracy confirms no-depth SCOT dominates across every joint. No-depth SCOT beats with-depth SCOT on all 17 joints on COCO, with the largest gains on the joints where depth should theoretically help most: shoulders (+6.0/+6.2), hips (+6.1/+6.2), knees (+5.0/+5.5). The depth signal is so unreliable on COCO that even joints where 3D separation would be most informative are better served by a 2D-only model with hard type constraints.

Experiment 13 — Residual SCOT (spatial prior + learned correction)

Motivation. SCOT beats kNN on synthetic data but loses on COCO. One hypothesis for why: kNN is a local, greedy method — each keypoint independently picks its nearest cluster centre. On easy scenes where people are well-separated, greedy is optimal. OT is a global method — it finds the best overall assignment considering all keypoints simultaneously. That global reasoning helps on hard scenes (overlapping people) but on easy scenes it introduces noise from the Sinkhorn entropy regularisation, which smears assignments that should be sharp.

The idea: decompose the cost matrix into a spatial prior and a learned correction:

$$C_{\text{final}} = C_{\text{spatial}} + \lambda \cdot C_{\text{learned}}$$

C_{spatial} is the squared distance from each keypoint to K spatial centroids computed via k-means on 2D positions (detached, not learned). This encodes geometric proximity — equivalent to what kNN uses. C_{learned} is the squared distance from encoded GAT embeddings to learned prototypes — the same cost as vanilla SCOT.

On easy scenes, C_{spatial} already has the right answer and the learned residual barely matters. On hard scenes, C_{spatial} is ambiguous and C_{learned} provides the discriminative correction. The spatial prior anchors easy cases, the learned cost handles hard cases.

This was implemented in a separate file (`ot_head_residual.py`) to preserve the original SCOT head for ablation. The λ parameter was set to 0.2, meaning the spatial cost contributes 80% and the learned cost 20%.

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.901	0.110	0.831	0.801	1.000
kNN (Residual SCOT GAT)	0.884	0.119	0.801	0.766	1.000
SCOT (no depth)	0.923	0.105	0.803	0.767	0.997
Residual SCOT (λ=0.2)	0.909	0.106	0.801	0.766	0.994

Table 1.12: Residual SCOT vs vanilla SCOT on synthetic test set.

Synthetic results. Per-joint comparison on synthetic data:

Joint	kNN	SCOT	Residual SCOT
nose	0.888	0.895	0.872
left eye	0.889	0.907	0.891
right eye	0.891	0.878	0.885
left ear	0.905	0.902	0.875
right ear	0.900	0.915	0.896
left shoulder	0.924	0.950	0.942
right shoulder	0.911	0.950	0.953
left elbow	0.900	0.965	0.945
right elbow	0.899	0.931	0.923
left wrist	0.905	0.939	0.908
right wrist	0.864	0.891	0.893
left hip	0.889	0.968	0.919
right hip	0.886	0.944	0.932
left knee	0.872	0.947	0.921
right knee	0.886	0.919	0.920
left ankle	0.818	0.897	0.911
right ankle	0.805	0.896	0.870

Table 1.13: Per-joint accuracy. Residual SCOT falls between kNN and SCOT on most joints.

Method	Synthetic	COCO	Transfer
kNN (no depth)	0.901	0.879	−2.2
SCOT (no depth)	0.923	0.862	−6.1
Residual SCOT (λ=0.2)	0.909	0.859	−5.0

Table 1.14: Transfer comparison. Residual SCOT's gap is slightly smaller but the difference is marginal.

COCO transfer results. Full COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.879	0.170	0.837	0.782	1.000
SCOT (no depth)	0.862	0.174	0.835	0.779	0.988
Residual SCOT (λ=0.2)	0.859	0.177	0.828	0.770	0.990

Table 1.15: COCO val2017 results.

Analysis. 1. Residual SCOT beats kNN but underperforms vanilla SCOT on synthetic data. At 0.909, Residual SCOT is 2.5 points above kNN (0.884) but 1.4 points below SCOT (0.923). The spatial prior anchors assignments toward k-means centroids, which helps on easy scenes but constrains the learned cost from making the corrections that SCOT makes on hard scenes. The per-joint pattern confirms this — SCOT gets 0.968 on left hip vs 0.919 for Residual SCOT, 0.965 on left elbow vs 0.945. The spatial prior is holding back the learned signal on exactly the joints where SCOT excels.

2. On COCO the difference is negligible. Residual SCOT (0.859) is within noise of SCOT (0.862). The transfer gap is slightly smaller (-5.0 vs -6.1), which was the hypothesis — the spatial prior should be more robust to domain shift. But the improvement is marginal and not worth the added complexity (k-means centroid computation per forward pass, extra hyperparameter λ).

3. The spatial prior doesn't help because it's redundant. The k-means centroids computed from 2D positions encode the same information that the kNN graph edges already provide. The GAT has already absorbed spatial proximity through message passing — the contrastive loss then separates the embeddings by person identity. Adding spatial centroids back into the cost matrix is giving the model information it already has, weighted against the learned signal that provides something new.

4. Vanilla SCOT is the better formulation. The learned cost from GAT embeddings is the right signal on its own. The spatial prior dilutes it without meaningful benefit on either synthetic or COCO data. This is a useful ablation result — it confirms that SCOT's performance comes from the learned cost and hard type constraints, not from spatial proximity which kNN already captures optimally.

Experiment 14 — Adaptive SCOT (scene-predicted temperature)

Motivation. The Sinkhorn temperature τ controls how "soft" the transport plan is. High τ gives smooth, spread-out assignments (global OT reasoning). Low τ approaches hard, greedy assignment (kNN-like). Vanilla SCOT fixes $\tau = 0.1$, which may not be optimal for all scenes — easy scenes with well-separated people might benefit from lower τ (sharper), while hard scenes with overlapping people might benefit from higher τ (more global).

The adaptive variant predicts τ from the scene:

$$\tau = \sigma(\text{MLP}(\text{graph_summary})) \cdot (\tau_{\max} - \tau_{\min}) + \tau_{\min}$$

where the graph summary is computed from cost matrix statistics (mean, standard deviation, row-wise entropy) and mean-pooled GAT embeddings. The model learns when to trust local assignments and when to invoke global reasoning. Implemented in a separate file (`ot_head_adaptive.py`) with $\tau_{\min} = 0.01$, $\tau_{\max} = 0.5$.

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.901	0.110	0.831	0.801	1.000
SCOT (fixed τ=0.1)	0.923	0.105	0.803	0.767	0.997
Residual SCOT (λ=0.2)	0.909	0.106	0.801	0.766	0.994
Adaptive SCOT	**0.928**	**0.098**	0.803	0.764	0.994

Table 1.16: Synthetic test set. Adaptive SCOT achieves the highest PGA of any method.

Method	Synthetic	COCO	Transfer
kNN (no depth)	0.901	0.879	−2.2
SCOT (fixed τ=0.1)	0.923	0.862	−6.1
Residual SCOT (λ=0.2)	0.909	0.859	−5.0
Adaptive SCOT	0.928	0.863	−6.5

Table 1.17: Transfer comparison across all SCOT variants.

Results. COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.879	0.170	0.837	0.782	1.000
SCOT (fixed τ=0.1)	0.862	0.174	0.835	0.779	0.988
Residual SCOT (λ=0.2)	0.859	0.177	0.828	0.770	0.990
Adaptive SCOT	0.863	0.176	0.832	0.775	0.991

Table 1.18: COCO val2017 results.

Per-joint accuracy on synthetic data:

Joint	kNN	SCOT	Adaptive SCOT
nose	0.891	0.895	0.926
left eye	0.903	0.907	0.906
right eye	0.904	0.878	0.918
left ear	0.919	0.902	0.935
right ear	0.897	0.915	0.949
left shoulder	0.910	0.950	0.963
right shoulder	0.910	0.950	0.934
left elbow	0.921	0.965	0.942
right elbow	0.887	0.931	0.949
left wrist	0.906	0.939	0.925
right wrist	0.874	0.891	0.876
left hip	0.881	0.968	0.956
right hip	0.882	0.944	0.953
left knee	0.862	0.947	0.922
right knee	0.870	0.919	0.925
left ankle	0.809	0.897	0.901
right ankle	0.817	0.896	0.893

Table 1.19: Per-joint accuracy. Adaptive SCOT improves on some joints, trades off on others.

Analysis. **1. Adaptive SCOT achieves the highest synthetic PGA (0.928).** A small improvement over vanilla SCOT (0.923, +0.5 points). The best checkpoint is at epoch 15 — even faster convergence than vanilla SCOT (epoch 30). The scene-adaptive temperature is providing marginal benefit on synthetic data where the tau predictor can learn the difficulty distribution.

2. On COCO, adaptive SCOT (0.863) is essentially identical to vanilla SCOT (0.862). The adaptive temperature does not help with transfer. The tau predictor was trained on synthetic scene difficulty patterns and does not generalise to real data where the difficulty distribution is different. The same face-joint weakness persists (ears 0.711/0.787 vs kNN 0.851/0.878).

3. The transfer gap is slightly larger (-6.5 vs -6.1). The adaptive tau adds a learned component (the MLP predictor) that is sensitive to distributional shift. On synthetic data this component helps; on COCO it is miscalibrated and slightly hurts.

SCOT ablation summary

SCOT variant	Synthetic	COCO	What it changes
Vanilla (fixed τ=0.1)	0.923	0.862	Baseline
Residual (spatial prior)	0.909	0.859	Adds spatial cost $\rightarrow$ hurts
Adaptive (learned τ)	0.928	0.863	Predicts $\tau \rightarrow$ marginal synthetic gain

Table 1.20: SCOT ablation. Vanilla SCOT is the best balance of performance and simplicity.

The conclusion across all three variants: **vanilla SCOT with fixed $\tau = 0.1$ is the right choice**. The adaptive variant adds complexity for a 0.5 point synthetic gain that does not transfer. The residual variant actively hurts by diluting the learned cost with redundant spatial information. The core contribution — hard type constraints via structured optimal transport — is what matters, not the temperature parameterisation or cost decomposition.

Experiment 15 — Unbalanced SCOT (K-free assignment)

Motivation. All previous SCOT variants require the number of people K as input. In practice K is unknown at inference. Unbalanced optimal transport addresses this by relaxing the target marginal constraint via a KL divergence penalty. Instead of requiring every person-type slot to be filled, unused slots naturally receive near-zero assignment mass.

The formulation uses $K_{\max} = 20$ person templates (well above the expected maximum). The unbalanced Sinkhorn algorithm modifies the column normalisation step:

$$\text{standard: } \log v = \log c - \text{logsumexp}(\ldots)$$

$$\text{unbalanced: } \log v = \frac{\rho}{\rho + \tau} \cdot (\log c - \text{logsumexp}(\ldots))$$

The scaling factor $\rho/(\rho+\tau) < 1$ dampens column normalisation, allowing target slots to deviate from their marginal. As $\rho \rightarrow \infty$, standard Sinkhorn is recovered. The number of people emerges from the solution: person k exists if $\sum_i T_{i,(k,\cdot)} >$ threshold.

Implemented in `ot_head_unbalanced.py` with $\rho = 1.0$, $K_{\max} = 20$, 20 Sinkhorn iterations, person threshold 0.5. During training, ground truth K is still provided so the loss operates on the correct number of logit columns.

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.901	0.110	0.831	0.801	1.000
SCOT (vanilla)	0.923	0.105	0.803	0.767	0.997
Unbalanced SCOT	0.886	0.123	0.826	0.792	0.979

Table 1.21: Synthetic test set. Unbalanced SCOT underperforms vanilla SCOT.

Method	Synthetic	COCO	Transfer
kNN (no depth)	0.901	0.879	−2.2
SCOT (vanilla)	0.923	0.862	−6.1
Residual SCOT	0.909	0.859	−5.0
Adaptive SCOT	0.928	0.863	−6.5
Unbalanced SCOT	0.886	0.842	−4.4

Table 1.22: Full SCOT variant comparison.

Results. COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.879	0.170	0.837	0.782	1.000
SCOT (vanilla)	0.862	0.174	0.835	0.779	0.988
Unbalanced SCOT	0.842	0.203	0.828	0.770	0.921

Table 1.23: COCO val2017 results. Unbalanced SCOT is the weakest variant.

Analysis. 1. Unbalanced SCOT (0.886) does not beat kNN on synthetic data. It falls 3.7 points below vanilla SCOT (0.923). The training is noticeably noisier — the head loss oscillates between 0.3 and 1.5 in the first 20 epochs, and the val PGA bounces between 0.70 and 0.86 without stabilising. With 20 person templates and only 2–5 people per scene, the solver has too many degrees of freedom. The KL relaxation at $\rho = 1.0$ allows person slots to partially empty even when they should be filled, weakening the assignment.

2. On COCO (0.842) it is the worst SCOT variant. Detection F1 at 0.921 confirms the unbalanced solver gets person count wrong more often. Even though ground truth K is provided during evaluation, the relaxed target marginal distributes mass less sharply than vanilla Sinkhorn.

3. Vanilla SCOT's tight constraints are an advantage, not a limitation. Knowing K and enforcing exact marginals constrains the problem, which means fewer solutions to search over and more reliable convergence. Every variant that adds flexibility (adaptive τ, spatial prior, unbalanced marginals) trades accuracy for generality that does not pay off in the current evaluation setup.

Updated SCOT ablation summary

SCOT variant	Synthetic	COCO	What it changes
Vanilla (fixed τ=0.1)	0.923	0.862	Baseline
Residual (spatial prior)	0.909	0.859	Adds spatial cost → hurts
Adaptive (learned τ)	0.928	0.863	Predicts τ → marginal synthetic gain
Unbalanced (K-free)	0.886	0.842	Relaxed marginals → hurts significantly

Table 1.24: Complete SCOT ablation. Vanilla SCOT remains the best formulation.

Vanilla SCOT remains the strongest formulation so far. The ablation shows that the core contribution is the type-constrained structured assignment via OT, not the specific parameterisation of temperature, cost, or marginal constraints.

Experiment 16 — Dustbin SCOT (K-free via explicit rejection)

Motivation. The unbalanced approach relaxed target marginals via KL penalty, which made assignments noisy. An alternative for K-free inference: keep standard balanced Sinkhorn (sharp assignments) but add an explicit dustbin column that competes with person slots. Keypoints whose best person assignment costs more than the dustbin cost go to the dustbin instead. After solving, person slots with significant mass are active; the rest are empty.

This is the approach used by DETR for object detection — a learned "no object" class that absorbs unused queries. The key difference from unbalanced OT: the dustbin is an explicit competing option with a learned cost, not a relaxation of the marginal constraints.

Implemented in `ot_head_dustbin.py` with $K_{\max} = 20$, learned dustbin cost initialised at 1.0, person threshold 0.5. During training, GT K is provided for the logit columns (same as unbalanced).

Method	Synthetic	COCO	Transfer
kNN (no depth)	0.901	0.879	−2.2
SCOT (vanilla)	0.923	0.862	−6.1
Unbalanced SCOT	0.886	0.842	−4.4
Dustbin SCOT	0.861	0.832	−2.9

Table 1.25: K-free variant comparison.

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.901	0.110	0.831	0.801	1.000
SCOT (vanilla)	0.923	0.105	0.803	0.767	0.997
Dustbin SCOT	0.861	0.134	0.810	0.775	0.969

Table 1.26: Synthetic test set. Dustbin SCOT underperforms kNN.

Results. COCO results:

Method	PGA	Std	NMI	ARI	Det F1
kNN (no depth)	0.879	0.170	0.837	0.782	1.000
SCOT (vanilla)	0.862	0.174	0.835	0.779	0.988
Unbalanced SCOT	0.842	0.203	0.828	0.770	0.921
Dustbin SCOT	0.832	0.200	0.835	0.779	0.967

Table 1.27: COCO val2017. Dustbin has better Det F1 than unbalanced but lower PGA.

Analysis. 1. Dustbin SCOT (0.861) does not beat kNN on synthetic data. The dustbin absorbs too many keypoints that should be assigned to persons. With 20 person templates during training (even though GT K is provided for the logits), the cost matrix has $20 \times 17 = 340$ slots plus dustbin competing for $\sim$60 keypoints. The dustbin provides an easy path to minimising loss: absorb ambiguous keypoints rather than resolving them.

2. Better than unbalanced on Detection F1 (0.967 vs 0.921). Balanced Sinkhorn with an explicit dustbin gives sharper assignments than KL-relaxed marginals. The dustbin cleanly separates "assigned to a person" from "unassigned" rather than smearing mass across partially-filled slots.

3. The transfer gap is the smallest of any SCOT variant (-2.9). The dustbin acts as a natural regulariser — by absorbing the hardest keypoints during training, the remaining assignments are more confident and transfer better. But the overall PGA is too low for this to be useful.

4. Both K-free approaches share the same fundamental problem. With 20 templates and only 2–5 people, the solver wastes capacity on empty slots. The prototype pool has 20 learned vectors but only 2–5 are meaningful; the rest are noise that the model must learn to ignore. Vanilla SCOT with exact K avoids this entirely — it creates exactly K slots and all must be filled.

Final SCOT ablation summary

SCOT variant	Synthetic	COCO	What it changes
Vanilla (fixed τ=0.1)	**0.923**	**0.862**	Baseline — best overall
Residual (spatial prior)	0.909	0.859	Adds spatial cost $\rightarrow$ hurts
Adaptive (learned τ)	0.928	0.863	Predicts $\tau \rightarrow$ marginal synthetic gain
Unbalanced (KL-relaxed)	0.886	0.842	Relaxed marginals $\rightarrow$ hurts
Dustbin (explicit rejection)	0.861	0.832	Dustbin absorbs too much $\rightarrow$ hurts

Table 1.28: Complete SCOT ablation across all five variants.

Vanilla SCOT is the strongest formulation. Every variant that adds flexibility — adaptive temperature, spatial prior, relaxed marginals, explicit dustbin — trades accuracy for generality that does not pay off. The core contribution is the type-constrained structured assignment via OT with known K, not the parameterisation details.

K-free inference remains a valuable direction for future work. Both approaches tested (unbalanced and dustbin) suffer from the same root cause: too many unused prototype slots dilute the optimisation. A more promising direction would be predicting K first via a lightweight count head, then running vanilla SCOT with the predicted K. This preserves the tight constraints that make vanilla SCOT work while removing the dependency on ground truth person count. Alternatively, a curriculum that starts training with known K and gradually introduces extra slots could teach the model to concentrate mass on active prototypes before being asked to handle unknown K.

COP-Kmeans — Type-Constrained K-Means

Motivation

SCOT demonstrated that the type constraint is the key contribution — hard enforcement that no two keypoints of the same type can be assigned to the same person. But SCOT wraps this constraint inside a learned optimal

transport formulation with person prototypes, Sinkhorn iterations, and supervised training. What if the constraint is applied directly to the kNN baseline instead?

COP-Kmeans (Wagstaff et al. 2001) extends standard k-means with **must-not-link constraints**: pairs of points that are forbidden from being in the same cluster. For pose grouping, the constraints are simple: any two keypoints of the same type cannot be in the same cluster. A left elbow and another left elbow must be in different person clusters.

This is the minimal possible intervention on the kNN baseline — the same algorithm, the same embeddings, the same K, with one additional constraint. No training required. If the type constraint is the reason SCOT works, COP-Kmeans should capture most of the benefit without any of the complexity.

How it works

The algorithm is identical to standard k-means with one change in the assignment step:

1. Initialise centroids with k-means++
2. **Constrained assignment:** sort keypoints by distance to nearest centroid (most confident first). For each keypoint, assign to the nearest centroid whose cluster does not already contain a keypoint of the same type. If the nearest centroid violates the constraint, try the next nearest, and so on.
3. **Update:** recompute centroids as cluster means
4. Repeat until convergence (10 random restarts, best kept)

The confidence-first ordering ensures that ambiguous keypoints (far from all centroids) are resolved after the easy ones are settled. No training is required — COP-Kmeans runs on the existing contrastive GAT embeddings at inference time using the same checkpoint as the kNN baseline.

Results — Synthetic test set

Method	PGA	Diff vs kNN
kNN (standard k-means)	0.901	—
COP-Kmeans	0.913	+1.2
SCOT	0.923	+2.2

Table 1.29: COP-Kmeans vs kNN and SCOT on synthetic test set.

The constraint changed 63.1% of assignments (3754 / 5950 keypoints). Per-joint accuracy:

Joint	kNN	COP-Kmeans	Diff
nose	0.871	0.926	+5.4
left eye	0.894	0.920	+2.6
right eye	0.900	0.926	+2.6
left ear	0.929	0.943	+1.4
right ear	0.903	0.943	+4.0
left shoulder	0.929	0.954	+2.6
right shoulder	0.920	0.943	+2.3
left elbow	0.923	0.957	+3.4
right elbow	0.874	0.931	+5.7
left wrist	0.894	0.900	+0.6
right wrist	0.863	0.871	+0.9
left hip	0.909	0.923	+1.4
right hip	0.914	0.911	−0.3
left knee	0.866	0.871	+0.6
right knee	0.877	0.897	+2.0
left ankle	0.840	0.783	−5.7
right ankle	0.860	0.840	−2.0

Table 1.30: Per-joint accuracy on synthetic data. Most joints improve; ankles slightly degrade.

Results — COCO val2017 (zero-shot transfer)

Method	PGA	Diff vs kNN
kNN (standard k-means)	0.879	—
SCOT (no depth)	0.862	−1.7
COP-Kmeans	**0.910**	**+3.1**

Table 1.31: COP-Kmeans on COCO — the best result in the entire project.

The constraint changed 57.4% of assignments (39165 / 68176 keypoints). Per-joint accuracy:

Joint	kNN	COP-Kmeans	Diff
nose	0.810	0.880	+7.0
left eye	0.797	0.871	+7.4
right eye	0.806	0.871	+6.5
left ear	0.758	0.816	+5.8
right ear	0.779	0.840	+6.0
left shoulder	0.734	0.808	+7.3
right shoulder	0.768	0.839	+7.1
left elbow	0.750	0.811	+6.1
right elbow	0.776	0.834	+5.8
left wrist	0.768	0.825	+5.6
right wrist	0.782	0.842	+6.0
left hip	0.778	0.861	+8.3
right hip	0.781	0.856	+7.6
left knee	0.787	0.846	+6.0
right knee	0.775	0.843	+6.7
left ankle	0.743	0.800	+5.7
right ankle	0.751	0.828	+7.7

Table 1.32: Per-joint accuracy on COCO. Every joint improves, +5.6 to +8.3 points.

Analysis

1. COP-Kmeans achieves the best COCO result in the entire project (0.910). This is 3.1 points above kNN (0.879) and 4.8 points above SCOT (0.862). No training was required — the same contrastive GAT checkpoint was used. The only change is one constraint during k-means assignment.

2. The type constraint is doing substantial work. On COCO, 57.4% of assignments were changed by the constraint. Standard k-means was routinely assigning duplicate types to the same person in more than half of all cases, and the constraint fixes this.

3. Every joint improves on COCO. Unlike SCOT which lost on face joints (ears -8 to -14 points vs kNN), COP-Kmeans improves across the board — +5.6 to +8.3 points on every joint. The largest gains are on hips (+8.3/+7.6) and the nose (+7.0).

4. COP-Kmeans transfers almost perfectly. The transfer gap is -0.3 points (0.913 synthetic $\rightarrow$ 0.910 COCO) — essentially zero. Compare this to SCOT's -6.1 point gap. COP-Kmeans has no learned parameters beyond the GAT, so there is nothing to overfit to the synthetic distribution. The type constraint is domain-invariant by construction — it encodes skeleton structure, not learned features.

5. COP-Kmeans vs SCOT: simplicity wins on transfer. SCOT beats COP-Kmeans on synthetic data (0.923 vs 0.913) because the learned cost function and Sinkhorn transport provide better global optimisation when embeddings are clean. But on COCO, COP-Kmeans wins decisively (0.910 vs 0.862) because it has no learned components that can overfit.

Method	Synthetic	COCO	Transfer gap
kNN	0.901	0.879	−2.2
SCOT	0.923	0.862	−6.1
COP-Kmeans	**0.913**	**0.910**	**−0.3**

Table 1.33: Transfer comparison. COP-Kmeans has the smallest sim-to-real gap of any method.

6. The ankle regression on synthetic data (−5.7 left ankle). When confident keypoints are assigned first, ankles — which are often the most spatially ambiguous — are assigned last and may be forced to a suboptimal centroid by accumulated constraints. On COCO this does not occur because there are more keypoints per scene and the constraints are less restrictive relative to available slots.

7. This is the result that DMoN was supposed to produce. The original SA-DMoN hypothesis was that domain-invariant structural signal (modularity) would improve sim-to-real transfer. Ten SA-DMoN experiments failed to deliver this. COP-Kmeans achieves it with a single constraint: no duplicate joint types per cluster. The structural signal that transfers across domains is not graph topology — it is the skeleton's type structure, enforced as a hard constraint on the assignment algorithm rather than a soft loss on a learned clustering head.

Type-Constrained Agglomerative Clustering

Motivation

COP-Kmeans showed that type constraints on k-means produce strong results. A natural question: does the same constraint help with a different base algorithm? Agglomerative (bottom-up) clustering starts with each keypoint as its own cluster and iteratively merges the two closest clusters. Adding the type constraint means: never merge two clusters that would create duplicate joint types.

This has a potential advantage over COP-Kmeans: it doesn't need K at inference. Standard agglomerative clustering stops at a target K, but type-constrained agglomerative could stop when all remaining merges would violate constraints — K emerges from the data. For this evaluation, GT K is provided for fair comparison, but the method could work without it.

How it works

1. Start with N clusters (one per keypoint)
2. Find the closest pair of clusters (average linkage — mean pairwise distance between members)
3. If merging would create duplicate joint types in the merged cluster, skip this pair and try the next closest
4. Merge the closest valid pair
5. Repeat until K clusters remain

No training required — runs on the same contrastive GAT embeddings as kNN and COP-Kmeans.

Results

Method	Synthetic	COCO	Transfer gap
kNN	0.901	0.879	−2.2
Agglomerative	0.837	0.887	+5.0
COP-Kmeans	0.913	0.910	−0.3

Table 1.34: Agglomerative clustering comparison.

Per-joint accuracy on COCO:

Joint	kNN	Agglomerative	Diff
nose	0.810	0.841	+3.1
left eye	0.797	0.847	+5.0
right eye	0.806	0.846	+4.0
left ear	0.758	0.802	+4.4
right ear	0.779	0.814	+3.4
left shoulder	0.734	0.775	+4.1
right shoulder	0.768	0.796	+2.8
left elbow	0.750	0.764	+1.4
right elbow	0.776	0.790	+1.4
left wrist	0.768	0.782	+1.4
right wrist	0.782	0.793	+1.2
left hip	0.778	0.793	+1.5
right hip	0.781	0.801	+2.0
left knee	0.787	0.788	+0.1
right knee	0.775	0.779	+0.4
left ankle	0.743	0.752	+0.9
right ankle	0.751	0.763	+1.2

Table 1.35: Every joint improves on COCO, but gains are smaller than COP-Kmeans.

Analysis

1. Agglomerative hurts on synthetic (-6.4) but helps on COCO ($+0.7$). The type constraint provides transfer benefit (every joint improves on COCO) but the bottom-up merging strategy is fundamentally weaker than k-means on clean embeddings. On synthetic data where the embeddings are well-separated, k-means finds the right clusters immediately; agglomerative makes irrevocable merge decisions that accumulate errors.

2. The irrevocable-merge problem. Agglomerative clustering cannot undo bad decisions. If it merges the wrong pair early (because they are the closest valid pair), every subsequent merge is conditioned on that error.

K-means can recover from bad assignments by recomputing centroids on the next iteration. This is why COP-Kmeans (0.913/0.910) dramatically outperforms agglomerative (0.837/0.887) despite using the same type constraint.

3. The COCO gains are smaller than COP-Kmeans. Agglomerative improves kNN by +0.7 on COCO; COP-Kmeans improves it by +3.1. Per-joint gains are +1 to +5 for agglomerative vs +5.6 to +8.3 for COP-Kmeans. The type constraint is the same in both methods — the difference is entirely in the base clustering algorithm.

4. The positive transfer gap ($+5.0$) **is misleading.** Agglomerative goes from 0.837 synthetic to 0.887 COCO, which looks like it transfers well. But this is because it performs so poorly on synthetic data (where the problem is easy) that the COCO result is higher by comparison. COP-Kmeans at -0.3 transfer gap with much higher absolute numbers on both datasets is the genuinely good transfer result.

5. Conclusion: the base algorithm matters as much as the constraint. The type constraint helps both methods on COCO, confirming it is a genuinely useful inductive bias for sim-to-real transfer. But COP-Kmeans is the right vehicle for it — k-means' iterative centroid refinement is a better assignment mechanism than agglomerative's greedy bottom-up merging, and the constraint integrates cleanly into the assignment step without the irrevocable-decision problem.

Skeleton-Aware Spectral Clustering

Motivation

The third type-constrained variant: apply the constraint to spectral clustering. Build an affinity matrix from GAT embedding similarity (RBF kernel on cosine distance), then zero out affinities between same-type keypoints. Spectral clustering computes eigenvectors of the modified graph Laplacian and runs k-means in the spectral embedding space. The type constraint lives in the graph structure — same-type keypoints are disconnected in the affinity graph, so the Laplacian naturally separates them.

This is more mathematically principled than COP-Kmeans — the constraint modifies the graph that spectral clustering reasons about, rather than being a greedy check during assignment. No training required.

Results

Method	Synthetic	COCO
kNN	0.901	0.879
Spectral (type-constrained)	0.915	0.894

Table 1.36: Skeleton-aware spectral clustering results.

Per-joint accuracy on synthetic:

Joint	kNN	Spectral	Diff
nose	0.871	0.897	+2.6
left eye	0.894	0.903	+0.9
right eye	0.900	0.914	+1.4
left ear	0.929	0.931	+0.3
right ear	0.903	0.931	+2.9
left shoulder	0.929	0.951	+2.3
right shoulder	0.920	0.940	+2.0
left elbow	0.923	0.937	+1.4
right elbow	0.874	0.903	+2.9
left wrist	0.894	0.906	+1.1
right wrist	0.863	0.883	+2.0
left hip	0.909	0.931	+2.3
right hip	0.914	0.940	+2.6
left knee	0.866	0.920	+5.4
right knee	0.877	0.900	+2.3
left ankle	0.840	0.840	+0.0
right ankle	0.860	0.854	−0.6

Table 1.37: Per-joint on synthetic. Every joint improves except ankles (neutral).

Per-joint accuracy on COCO:

Joint	kNN	Spectral	Diff
nose	0.810	0.848	+3.8
left eye	0.797	0.840	+4.3
right eye	0.806	0.845	+3.9
left ear	0.758	0.796	+3.8
right ear	0.779	0.810	+3.1
left shoulder	0.735	0.784	+5.0
right shoulder	0.768	0.809	+4.1
left elbow	0.750	0.785	+3.5
right elbow	0.776	0.807	+3.2
left wrist	0.768	0.788	+1.9
right wrist	0.782	0.815	+3.3
left hip	0.778	0.814	+3.6
right hip	0.780	0.829	+4.9
left knee	0.787	0.821	+3.4
right knee	0.775	0.813	+3.9
left ankle	0.743	0.774	+3.1
right ankle	0.751	0.777	+2.6

Table 1.38: Every joint improves on COCO, +1.9 to +5.0 points.

Analysis

1. Spectral clustering is the second-best type-constrained method. It beats kNN on both synthetic (+1.4) and COCO (+1.5), with every joint improving on both datasets. On synthetic it slightly edges COP-Kmeans (0.915 vs 0.913). On COCO, COP-Kmeans wins (0.910 vs 0.894).

2. The affinity zeroing approach is clean but causes numerical issues. On COCO, zeroing out same-type affinities frequently disconnects the graph, producing "graph is not fully connected" warnings from scikit-learn. The eigenvector computation becomes unstable on disconnected graphs, and some scenes require fallback to standard k-means. This does not happen on synthetic data where the embeddings are cleaner and fewer same-type pairs have high affinity to begin with.

3. COP-Kmeans handles the constraint more gracefully. COP-Kmeans integrates the type constraint into the assignment step of an iterative algorithm — if the nearest centroid is blocked, try the next one. Spectral clustering encodes the constraint in the affinity matrix before the algorithm runs, which is more principled but less robust when the modified graph has poor connectivity. The iterative approach is more forgiving of edge cases.

Final comparison across all type-constrained methods

Method	Synthetic	COCO	Transfer gap	Constraint mechanism
kNN (baseline)	0.901	0.879	−2.2	None
Agglomerative	0.837	0.887	+5.0	Block merges
Spectral	0.915	0.894	−2.1	Zero affinities
COP-Kmeans	**0.913**	**0.910**	**−0.3**	**Block assignments**
SCOT	0.923	0.862	−6.1	Infinite cost

Table 1.39: All type-constrained methods compared. COP-Kmeans is the best overall.

The type constraint improves every method on COCO regardless of how it is applied — agglomerative (+0.8), spectral (+1.5), COP-Kmeans (+3.1). This confirms that the skeleton type structure is a genuinely useful inductive bias for sim-to-real transfer in pose grouping.

COP-Kmeans is the best vehicle for the constraint: it achieves the highest COCO PGA (0.910), the smallest transfer gap (−0.3), and is the simplest to implement (one extra check in the k-means assignment step). SCOT achieves the highest synthetic PGA (0.923) through learned optimal transport but its transfer gap (−6.1) means it underperforms COP-Kmeans on real data by 4.8 points.

The conclusion across all methods in this project: **the type constraint is the contribution, and the simpler the mechanism for enforcing it, the better it transfers.**

SA-GAT — Skeleton-Aware Graph Attention Network

Motivation

All experiments so far modified the *grouping method* on top of the GAT embeddings. COP-Kmeans showed that a simple type constraint on k-means beats every learned head. But the embeddings themselves are still produced by a standard GAT that treats all edges identically — a left-shoulder-to-left-elbow edge gets the same attention function as a left-shoulder-to-right-ankle edge. These edges carry fundamentally different information for grouping.

The idea: make the GAT backbone itself skeleton-aware so that the embeddings are inherently better for grouping, regardless of which downstream method is used. This is an independent contribution from the grouping constraint — SA-GAT improves the embeddings, COP-Kmeans improves the assignment. Together they should compose.

Three modifications

Each modification is independently configurable for ablation.

Modification 1: Type-Pair Attention Bias. Categorise every edge in the graph based on the joint types at each end:

- `SAME_TYPE` — both keypoints are the same type (e.g. two left elbows). These *must* belong to different people. The attention should learn inter-person discrimination.
- `SKELETAL_NEIGHBOR` — directly connected on the COCO skeleton (e.g. shoulder–elbow). Strongest cue for same-person grouping — anatomically connected joints from the same person have consistent spatial relationships.

- `SAME_LIMB` — on the same limb but not directly adjacent (e.g. shoulder–wrist). Medium-range structural information.
- `CROSS_BODY` — unrelated body parts (e.g. left wrist–right ankle). Weak or no structural prior.

Each edge gets a one-hot category encoding [4] which is passed through an MLP and added as an attention bias via GATv2Conv's `edge_dim` mechanism. This lets the attention function weight different categories of interactions differently without replacing the core GATv2Conv implementation.

Modification 2: Skeleton-Relative Position Encoding. Standard GAT uses no explicit spatial information in attention — it is purely feature-based. Three edge features are computed and encoded as an additional attention bias:

- Spatial distance $\|pos_i - pos_j\|$ between the two keypoints
- Skeleton hop distance between their types (normalised), from the COCO skeleton graph
- Same-type indicator (1 if same type, 0 otherwise)

These are concatenated with the category one-hot and passed through the same edge encoder MLP. The skeleton hop distance tells the attention how anatomically related two keypoint types are, regardless of their spatial proximity.

Modification 3: Same-Type Repulsion Heads. In a multi-head attention setup, dedicate one or more heads exclusively to same-type neighbor interactions. Standard heads attend to all neighbors. The repulsion head only receives edges connecting same-type keypoints — all other edges are masked out.

This forces the repulsion head to explicitly process the inter-person signal: "I have two left elbows — how should their embeddings differ?" The contrastive loss then pushes the repulsion head to produce discriminative features for distinguishing same-type keypoints belonging to different people.

Implementation: the SA-GAT layer splits into a standard GATv2Conv (3 heads, all edges) and a repulsion GATv2Conv (1 head, same-type edges only). Their outputs are concatenated, giving the same total dimension as 4 standard heads.

Implementation

SA-GAT (`sa_gat.py`) is a drop-in replacement for the standard `GATEmbedding` — same forward signature, same output shape $[N, 128]$. All downstream grouping methods work without changes.

The key design decision: build on top of PyG's optimised GATv2Conv rather than reimplementing attention from scratch. The skeleton-aware modifications are injected as edge features (`edge_attr`) that GATv2Conv adds as attention biases. This preserves numerical stability and performance while adding the skeleton-aware signal. An initial implementation with custom scatter-based attention failed catastrophically (PGA 0.60) due to numerical instability in the manual softmax — switching to GATv2Conv with edge features fixed this immediately.

Config flags for ablation:

- `use_type_pair_attention`: category-specific attention bias
- `use_position_encoding`: spatial + skeleton hop encoding
- `use_repulsion_heads`: dedicated same-type heads
- `n_repulsion_heads`: how many heads reserved (default 1 of 4)

The ablation table:

	Modifications			Synthetic		COCO	
Config	Type-pair	Pos enc	Repulsion	kNN	COP-KM	kNN	COP-KM
Standard GAT	no	no	no	0.901	0.913	0.879	0.910
Type-pair only	yes	no	no	0.912	0.914	0.882	0.914
Pos-enc only	no	yes	no	0.905	0.917	0.882	0.916
Repulsion only	no	no	yes	0.905	0.912	0.878	0.908
SA-GAT full	**yes**	**yes**	**yes**	**0.910**	**0.925**	**0.882**	**0.915**

Table 1.40: SA-GAT ablation. All three modifications contribute; the full combination is best.

Results — SA-GAT full (all three modifications)

Trained with contrastive loss only (no grouping head), 50 epochs. Evaluated with kNN and COP-Kmeans.

Method	PGA	vs Standard GAT
Standard GAT + kNN	0.901	—
SA-GAT + kNN	0.910	+0.9
Standard GAT + COP-Kmeans	0.913	—
SA-GAT + COP-Kmeans	**0.925**	**+1.2**

Table 1.41: SA-GAT embeddings improve both kNN and COP-Kmeans on synthetic data.

Synthetic test set. SA-GAT + COP-Kmeans at 0.925 is the best kNN-based result in the project, above SCOT's 0.923.

Method	PGA	vs Standard GAT
Standard GAT + kNN	0.879	—
SA-GAT + kNN	0.882	+0.3
Standard GAT + COP-Kmeans	0.910	—
SA-GAT + COP-Kmeans	**0.915**	**+0.5**

Table 1.42: SA-GAT + COP-Kmeans on COCO — new project best (0.915).

COCO val2017 (zero-shot transfer). Per-joint accuracy on COCO (SA-GAT + COP-Kmeans):

Joint	SA-GAT kNN	SA-GAT + COP-KM	Diff
nose	0.815	0.882	+6.7
left eye	0.809	0.871	+6.3
right eye	0.819	0.887	+6.8
left ear	0.757	0.829	+7.2
right ear	0.785	0.853	+6.8
left shoulder	0.740	0.825	+8.5
right shoulder	0.751	0.818	+6.8
left elbow	0.756	0.815	+5.8
right elbow	0.761	0.817	+5.6
left wrist	0.772	0.830	+5.9
right wrist	0.774	0.840	+6.6
left hip	0.775	0.861	+8.6
right hip	0.780	0.858	+7.8
left knee	0.787	0.849	+6.3
right knee	0.785	0.847	+6.2
left ankle	0.774	0.840	+6.6
right ankle	0.780	0.838	+5.8

Table 1.43: Per-joint COCO accuracy. +5.6 to +8.6 improvement across every joint.

Analysis

1. SA-GAT + COP-Kmeans (0.915) is the new project best on COCO. It beats standard GAT + COP-Kmeans (0.910) by 0.5 points, and SCOT (0.862) by 5.3 points. The two contributions — better embeddings and type-constrained assignment — are additive and independent.

2. SA-GAT improves the embeddings themselves. kNN on SA-GAT (0.910 synthetic, 0.882 COCO) outperforms kNN on standard GAT (0.901, 0.879) on both datasets. The NMI improves from 0.831 to 0.848 on synthetic and from 0.837 to 0.850 on COCO. The skeleton-aware attention is producing more separable per-person embeddings.

3. The improvements compose with COP-Kmeans. SA-GAT gives +0.9 on synthetic with kNN. COP-Kmeans gives +1.2 on standard GAT. Together they give +2.4 (0.925 vs 0.901). On COCO: SA-GAT gives +0.3 with kNN, COP-Kmeans gives +3.1 on standard GAT, together +3.6 (0.915 vs 0.879). The contributions are independent because they operate at different levels — SA-GAT modifies the attention mechanism, COP-Kmeans modifies the assignment step.

4. The contrastive loss converges faster with SA-GAT. The loss reaches 0.037 at epoch 50, vs 0.050 for the standard GAT at the same point. The skeleton-aware edge features provide a stronger inductive bias that helps the contrastive loss separate people more efficiently.

5. SA-GAT + COP-Kmeans vs SCOT.

Method	Synthetic	COCO	Transfer gap
Standard GAT + kNN	0.901	0.879	−2.2
SCOT (learned OT head)	0.923	0.862	−6.1
Standard GAT + COP-Kmeans	0.913	0.910	−0.3
SA-GAT + COP-Kmeans	**0.925**	**0.915**	**−1.0**

Table 1.44: SA-GAT + COP-Kmeans is the best method on both datasets.

SA-GAT + COP-Kmeans beats SCOT on both synthetic and COCO. It achieves this without any learned grouping head — no prototypes, no Sinkhorn iterations, no supervised head loss. The skeleton awareness is split between the GAT backbone (attention modifications) and the grouping step (type constraint on k-means), both using domain knowledge rather than learned parameters. This is why it transfers better than SCOT: there are no learned components that can overfit to the synthetic distribution.

6. This defines the MEng/PEng split.

- **MEng contribution:** SA-GAT (skeleton-aware attention for multi-person pose) + type-constrained grouping (COP-Kmeans). Both are novel applications of the skeleton type structure as an architectural inductive bias. Neither requires a learned grouping head.
- **PEng contribution:** SCOT (type-constrained optimal transport). Novel formulation of pose grouping as structured assignment. Best synthetic result. Requires learned prototypes and Sinkhorn transport — more theoretically interesting, room to improve transfer with fine-tuning on real data.

SA-GAT ablation analysis

Each modification was tested in isolation (the other two disabled) to measure its individual contribution.

1. Type-pair attention is the strongest individual modification. kNN 0.912 on synthetic (vs 0.901 baseline) and COP-Kmeans 0.914 on COCO (vs 0.910). Learning different attention profiles for same-type vs skeletal-neighbor vs cross-body edges gives the model the strongest per-modification signal. Same-type edges (two left elbows from different people) are the most informative edge category for person discrimination, and having dedicated parameters for them is the key.

2. Position encoding provides the best COCO COP-Kmeans result of any single modification (0.916). Injecting spatial distance, skeleton hop distance, and the same-type indicator as edge features gives the attention mechanism geometric context it otherwise lacks. The skeleton hop distance is particularly valuable — it tells the model that a shoulder near an elbow (hop=1) is expected, while a shoulder near an ankle (hop=3+) is surprising.

3. Repulsion heads are the weakest individual modification. kNN 0.905 on synthetic (same as pos-enc) but COP-Kmeans 0.908 on COCO — slightly below the standard GAT + COP-Kmeans baseline of 0.910. Dedicating one of four attention heads exclusively to same-type edges reduces the capacity available for general message passing. With only 4 heads total, losing 25% of capacity hurts more than the specialisation gains. This might improve with more heads (e.g. 8 heads with 2 dedicated to repulsion).

4. The full combination is best. SA-GAT full (0.925 / 0.915) outperforms every individual modification. The three modifications are complementary: type-pair attention provides category-specific parameters, position encoding provides geometric context, and repulsion heads provide dedicated inter-person processing. Together they produce the best embeddings in the project.

SA-GAT + SCOT — Combining the Two Contributions

Motivation

SA-GAT produces better embeddings. SCOT provides better assignment via type-constrained optimal transport. These are independent contributions operating at different levels — SA-GAT modifies the attention mechanism, SCOT modifies the grouping step. The question: do they compose? If so, SA-GAT + SCOT could be the PEng system while SA-GAT + COP-Kmeans remains the MEng system.

Results

Method	Synthetic	COCO	Transfer gap
Standard GAT + kNN	0.901	0.879	−2.2
Standard GAT + SCOT	0.923	0.862	−6.1
Standard GAT + COP-KM	0.913	0.910	−0.3
SA-GAT + kNN	0.910	0.882	−2.8
SA-GAT + COP-KM	0.925	0.915	−1.0
SA-GAT + SCOT	0.924	0.863	−6.1
SA-GAT (SCOT-trained) + COP-KM	0.915	0.915	0.0

Table 1.45: Complete comparison across all backbone + grouping combinations.

SCOT head on COCO with SA-GAT backbone:

Joint	SA-GAT kNN	SA-GAT + SCOT	SA-GAT + COP-KM
nose	0.890	0.842	0.900
left eye	0.906	0.798	0.890
right eye	0.906	0.820	0.885
left ear	0.858	0.708	0.848
right ear	0.884	0.797	0.856
left shoulder	0.861	0.876	0.822
right shoulder	0.871	0.882	0.817
left elbow	0.847	0.816	0.816
right elbow	0.864	0.844	0.822
left wrist	0.850	0.795	0.832
right wrist	0.857	0.815	0.833
left hip	0.878	0.877	0.861
right hip	0.880	0.885	0.858
left knee	0.875	0.830	0.867
right knee	0.867	0.831	0.846
left ankle	0.863	0.815	0.840
right ankle	0.866	0.808	0.847

Table 1.46: Per-joint COCO accuracy. SCOT loses on face joints; COP-Kmeans wins overall.

Analysis

1. SA-GAT + SCOT (0.924) matches standard GAT + SCOT (0.923) on synthetic. The better embeddings provide minimal additional benefit to SCOT because SCOT's learned prototypes and Sinkhorn iterations already handle the assignment well on clean synthetic data. On synthetic data, the assignment mechanism is the bottleneck, not the embeddings.

2. On COCO, SA-GAT + SCOT (0.863) barely improves over standard GAT + SCOT (0.862). The transfer gap remains -6.1 regardless of the backbone. This confirms that SCOT's transfer problem is in the learned head — the prototypes and transport plan overfit to synthetic data — not in the embeddings. Better embeddings cannot fix a head that doesn't generalise.

3. COP-Kmeans on the SCOT-trained SA-GAT hits 0.915 on COCO. This is the same as COP-Kmeans on the contrastive-only SA-GAT (0.915). Joint training with SCOT does not degrade the SA-GAT embeddings — unlike standard GAT where the SCOT head loss slightly hurt the embeddings. The skeleton-aware attention is robust to the additional head loss gradient.

4. The SCOT per-joint pattern persists. SCOT beats kNN on shoulders and hips (where the type constraint helps) but loses badly on face joints (ears -8 to -18 points vs kNN). This pattern is identical to standard GAT + SCOT. The SA-GAT backbone does not fix SCOT's face-joint weakness because the problem is in the Sinkhorn assignment, not the embeddings.

5. COP-Kmeans dominates on COCO regardless of backbone. Every COP-Kmeans variant (0.910–0.915) outperforms every SCOT variant (0.862–0.863) on COCO. The type constraint applied as a hard check during

k-means assignment is more robust to domain shift than the type constraint applied as infinite cost in a learned optimal transport plan.

Final project summary

Method	Synthetic	COCO	Transfer	Notes
Baselines				
Standard GAT + kNN	0.901	0.879	−2.2	Baseline
Learned grouping heads (all failed to beat kNN on COCO)				
Slot attention	0.858	0.788	—	With depth
Graph partitioning	0.954	0.752	—	With depth
SA-DMoN (best)	0.805	0.817	—	No depth
Type-constrained methods				
Standard GAT + COP-KM	0.913	0.910	−0.3	No training needed
Standard GAT + SCOT	0.923	0.862	−6.1	Best synthetic head
SA-GAT combinations				
SA-GAT + kNN	0.910	0.882	−2.8	Better embeddings
SA-GAT + COP-KM	**0.925**	**0.915**	**−1.0**	**Best overall**
SA-GAT + SCOT	0.924	0.863	−6.1	SCOT transfer limits it

Table 1.47: All methods tested in this project. SA-GAT + COP-Kmeans is the best on both datasets.

The MEng contribution is SA-GAT (skeleton-aware attention with type-pair bias, position encoding, and repulsion heads) combined with COP-Kmeans (type-constrained k-means). Together they achieve 0.925 on synthetic and 0.915 on COCO — the best results in the project on both datasets, with a transfer gap of only -1.0 points. Neither component requires a learned grouping head. The skeleton type structure is encoded as an architectural inductive bias in the attention mechanism and as a hard constraint in the assignment algorithm.

The PEng direction is SCOT (type-constrained optimal transport) combined with SA-GAT. SCOT achieves 0.924 on synthetic — matching the MEng result — but its COCO transfer gap of -6.1 points is the main open problem. The path forward is either fine-tuning on real data, solving K estimation to remove the dependency on ground truth person count, or making the learned prototypes more robust to distributional shift. This builds directly on the MEng work: the same SA-GAT backbone, the same skeleton type structure, but a more ambitious (and publishable) assignment formulation.

K Estimation — Removing the Ground Truth K Dependency

The problem

Every method tested in this project receives the ground truth number of people K as input. kNN uses it as `n_clusters`, COP-Kmeans uses it identically, SCOT constructs $K \times 17$ typed slots. In a real production system K is unknown — the system receives detected keypoints and must determine both how many people are present and which keypoints belong to which person.

This is standard practice in bottom-up pose grouping research — most papers evaluate with GT keypoint locations and GT person count to isolate the grouping problem from detection. But solving K estimation would make the system complete and deployable.

Attempt 1: Eigenvalue gap

The classical spectral approach: build an affinity matrix from embedding similarity (RBF kernel), compute the graph Laplacian $L = D - W$, and estimate K from the largest gap between consecutive eigenvalues. The number of near-zero eigenvalues equals the number of connected components (people).

	SA-GAT	Standard GAT	SA-GAT (COCO)
Exact match	26%	27%	18%
Off by ≤ 1	58%	56%	54%

Table 1.48: Eigenvalue gap K estimation accuracy.

Results. Per GT K breakdown (SA-GAT, synthetic):

- K=2: 92% exact — near perfect
- K=3: 12% exact — mostly predicted K=2
- K=4: 0% exact — all predicted K=2 or K=3
- K=5: 0% exact — same failure

The eigengap is massively biased toward K=2. The contrastive loss produces embeddings where the first spectral gap (between eigenvalue 1 and 2) is always the largest — the strongest signal is "there are multiple people" (a binary split). The finer structure (3, 4, 5 people) produces smaller gaps that the simple argmax misses.

Grouping PGA with predicted K drops catastrophically: $0.910 \rightarrow 0.673$ on synthetic (-23.7 points). The eigengap approach is not viable.

Attempt 2: Learned K estimation head

A small MLP that predicts K from pooled GAT embeddings and the keypoint count:

$$K_{\text{pred}} = \text{MLP}(\text{mean_pool}(H) \,\|\, \text{max_pool}(H) \,\|\, N_{\text{keypoints}}/17)$$

The $N_{\text{keypoints}}/17$ feature is a strong prior — with 17 COCO joints per person, 85 keypoints almost certainly means 5 people. The MLP learns to refine this estimate using the embedding features (which encode how many distinct clusters exist). Trained jointly with the SA-GAT via L1 loss on the predicted K.

Results — Synthetic. K estimation: **100/100 exact match** on synthetic data (25/25 for each of K=2,3,4,5). Perfect. Grouping PGA with predicted K is identical to GT K (0.677 vs 0.677 for COP-Kmeans).

	Eigengap	Learned head
Exact match	18.3%	**55.5%**
Off by ≤ 1	53.7%	**80.4%**

Table 1.49: K estimation accuracy on COCO. Learned head is substantially better.

Results — COCO. Per GT K breakdown:

- K=1: 100% exact (1059/1059) — trivial case, N_keypoints is small
- K=2: 34% exact — underpredicts to K=1 (299/454)
- K=3: 20% exact — mostly predicts K=2 or K=1
- K=4+: <6% exact — systematic underprediction

The K head is much better than eigengap but still underpredicts on COCO for $K \geq 3$. The model was trained on synthetic data with $K = 2$–5; on COCO it sees up to $K = 13$ with different keypoint visibility patterns (more occlusion, more missing joints). The $N_{\text{keypoints}}/17$ prior is less reliable on COCO because occlusion means fewer visible joints per person.

The embedding quality problem. The K head achieved perfect K estimation on synthetic data but the grouping PGA was only 0.677 — dramatically below the SA-GAT without K head (0.910). The L1 loss on K prediction was **interfering with contrastive learning**. The mean-pooled embeddings were being optimised to encode person *count* rather than person *identity*. The embeddings became countable but not separable.

On COCO the same problem: kNN with GT K scored 0.793 (vs 0.882 for SA-GAT without K head). The K head loss corrupted the embeddings.

Attempt 3: Frozen backbone K head

The fix: freeze the SA-GAT weights and train only the K head on pre-computed embeddings. The SA-GAT checkpoint (0.910/0.882) is loaded and frozen — the K head learns to read K from embeddings that were already optimised for separation, without corrupting them.

Implemented in `train_k_head_frozen.py`: loads a GAT checkpoint, freezes all parameters, creates a fresh K head, and trains with L1 loss for 50 epochs on the virtual training set.

Results — Synthetic. K estimation: **100/100 exact match** (identical to joint training). Crucially, the embeddings are preserved: kNN PGA 0.910 and COP-Kmeans PGA 0.925, both identical to the SA-GAT without any K head. Grouping with predicted K matches grouping with GT K exactly (+0.000).

	Eigengap	Joint training	Frozen backbone
Exact match	18.3%	55.5%	**63.2%**
Off by ≤ 1	53.7%	80.4%	**83.5%**

Table 1.50: K estimation accuracy on COCO across all three approaches.

Results — COCO. Per GT K breakdown (frozen vs joint training):

- K=1: 97.6% (vs 100%) — slight regression, the frozen embeddings are optimised for multi-person separation not single-person detection
- K=2: **66%** (vs 34%) — nearly doubled, the discriminative embeddings make two-person scenes more readable
- K=3: **38%** (vs 20%) — also nearly doubled
- K=4: 11% (vs 5%) — improving but still low
- K=5+: <10% — systematic underprediction persists for large K

Method	GT K	Predicted K	Diff
kNN	0.882	0.848	−3.5
COP-Kmeans	0.915	0.849	−6.5

Table 1.51: COCO grouping PGA with frozen K head predictions.

Grouping with predicted K. COP-Kmeans drops more than kNN (−6.5 vs −3.5) because wrong K hurts constrained methods more — if K is too low, COP-Kmeans forces keypoints into too few clusters and the type constraint creates conflicts that wouldn't exist with the correct K.

Analysis. 1. Frozen backbone completely solves the embedding corruption problem. The SA-GAT embeddings are identical to the original checkpoint — kNN 0.910, COP-Kmeans 0.925 on synthetic. The joint training attempt degraded these to 0.613 and 0.677 respectively. Separating the K estimation objective from the contrastive objective is essential.

2. K estimation is substantially better with frozen embeddings. 63.2% exact (vs 55.5% joint, 18.3% eigengap) and 83.5% off-by-one. The discriminative embeddings produced by contrastive-only training contain clearer cluster structure that the K head can read.

3. The remaining error is from training distribution mismatch. The K head was trained on synthetic data with K=2–5. COCO has K=1–13+ with different occlusion patterns. K=1 (single person) is nearly perfect because N_keypoints is distinctively small. K=2 at 66% is reasonable. K≥4 fails because the model has never seen enough data in that range and the $N_{\text{keypoints}}/17$ prior becomes unreliable with occlusion.

4. COP-Kmeans with predicted K (0.849) does not yet beat kNN with GT K (0.879). The 3-point gap is entirely from K estimation errors. If K were perfect, COP-Kmeans would score 0.915. The path to closing this gap is training the K head on more diverse data (larger K range, real occlusion patterns) or fine-tuning on a small amount of COCO.

5. This is a viable PEng direction. The K head adds only ∼17K parameters, trains in 50 epochs on frozen embeddings (67 seconds total), and provides 83.5% off-by-one accuracy on COCO with zero COCO training data. With COCO fine-tuning or training on a wider K range, the predicted-K system could approach the GT-K performance and provide a complete, deployable pipeline: SA-GAT → K head → COP-Kmeans/SCOT, with no oracle required.

Training Infrastructure — GUID-based Output Management

The problem

During SCOT re-training, I discovered that re-running `trainer.py` with the same config overwrites the previous `best.pt` checkpoint. The original SCOT run achieved 0.923 synthetic PGA; after accidentally re-training with the same config (but only 50 epochs instead of 100), the checkpoint was overwritten with a 0.917 result. The original model was permanently lost.

This is a fundamental reproducibility problem. The old directory structure was:

```
outputs/checkpoints/{config_save_dir}/
    best.pt         # overwritten on every run
    final.pt
```

```
    history.json
```

There was a one-to-one mapping between config and output directory, so any re-run — whether intentional (parameter sweep) or accidental — destroyed the previous result. There was also no record of which exact config produced a given checkpoint, making it impossible to reproduce a run after the fact.

Solution: GUID-based run directories

Every training run now creates a unique subdirectory under `outputs/{config_name}/`:

```
outputs/
  train_scot_no_depth/
    a1b2c3d4/             # 8-char GUID
      best.pt
      final.pt
      history.json
      config.yaml          # full copy of the config used
      eval_results.json    # saved by evaluator
    e5f6g7h8/             # another run with same or different params
      ...
    latest -> a1b2c3d4    # symlink to most recent run
```

Key design decisions:

- The config name is derived from the YAML filename (e.g. `configs/train_scot_no_depth.yaml` → `train_scot_no_depth`). A `-name` override is available for re-running a saved `config.yaml` from inside a GUID directory, which would otherwise create a folder called "config".
- The full config YAML is copied into each run directory, so any result can be reproduced by pointing the trainer at that file: `python trainer.py -config outputs/.../config.yaml -name train_scot_no_depth`.
- A `latest` symlink is updated programmatically after each run, so quick evaluations can use `outputs/train_scot_no_depth/latest/best.pt` without knowing the GUID.
- Evaluation results are saved next to the checkpoint rather than to a global `outputs/eval_results.json` that was also being overwritten.

Previous frozen results from all experiments up to this point were preserved `outputs/frozen_checkpoints/`.

Appendix

Acronyms

$\longleftarrow \asymp \longrightarrow$

Bibliography